

Deep Learning

Lecture 3

Continuing our goal

- Image Classification:



This image by Nikita is
licensed under [CC-BY2.0](#)

—————→ cat

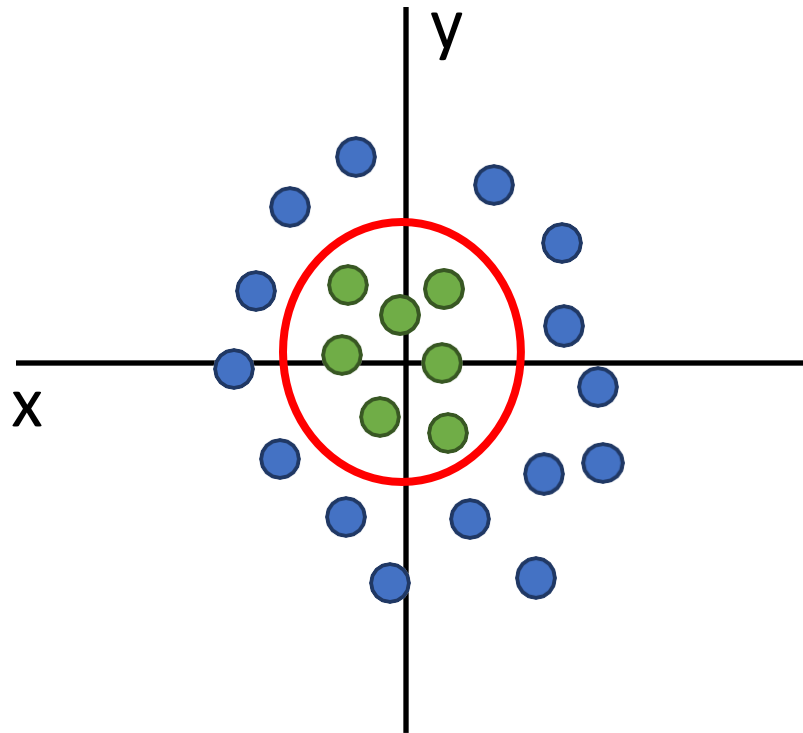
Linear Classifiers aren't that powerful

- One template per class: Can't recognize different modes of a class



Feature Transform

Original space



Nonlinear classifier
in original space!

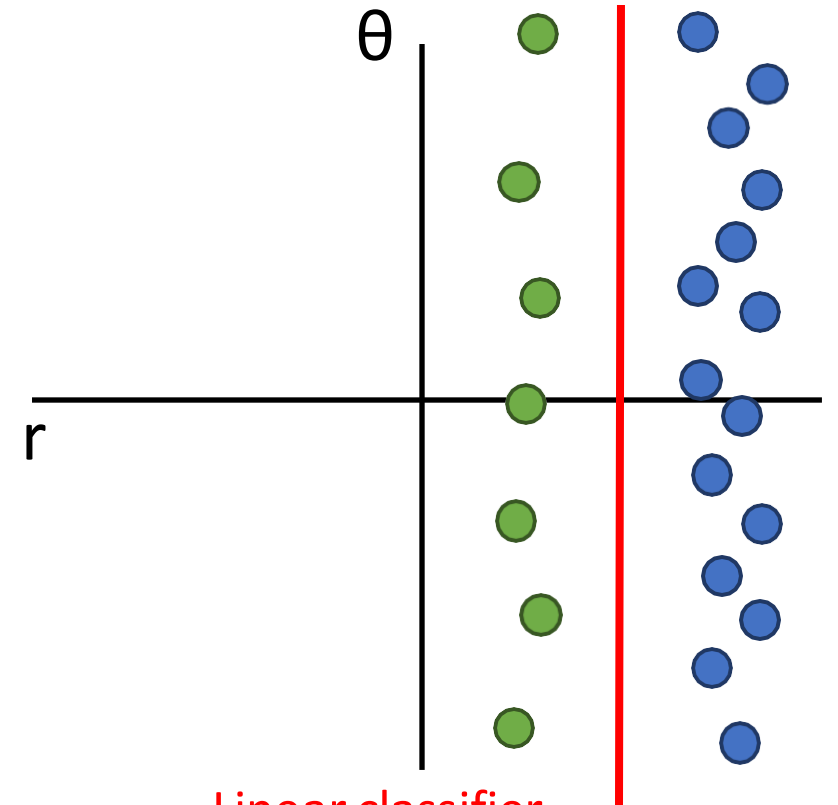
$$r = (x^2 + y^2)^{1/2}$$
$$\theta = \tan^{-1}(y/x)$$



Feature
transform

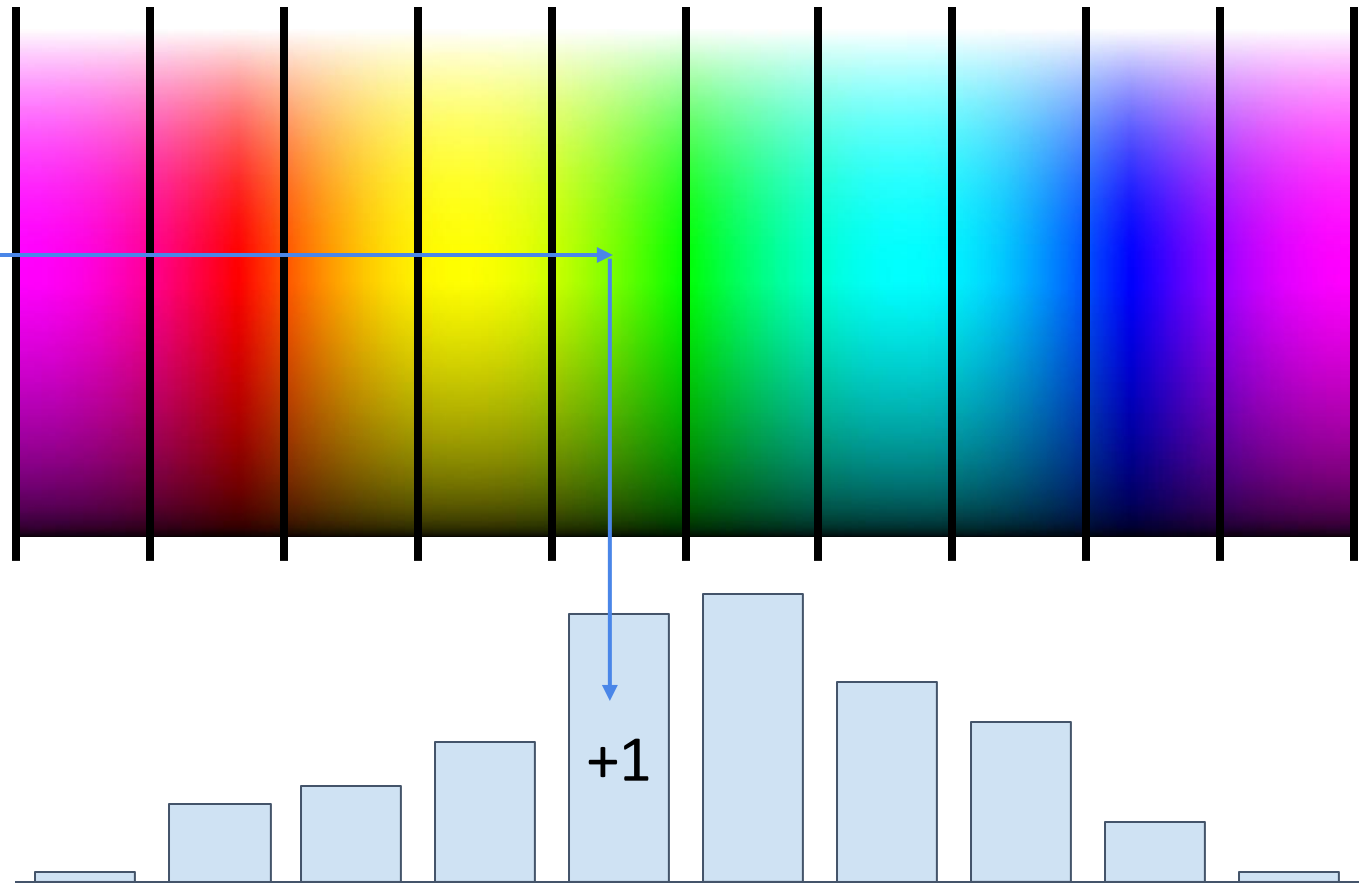


Feature space



Linear classifier
in feature space

Image Features: Color Histogram



Ignores texture,
spatial positions

Histogram of Oriented Gradients (HoG)



1. Compute edge direction / strength at each pixel
2. Divide image into 8x8 regions
3. Within each region compute a histogram of edge directions weighted by edge strength

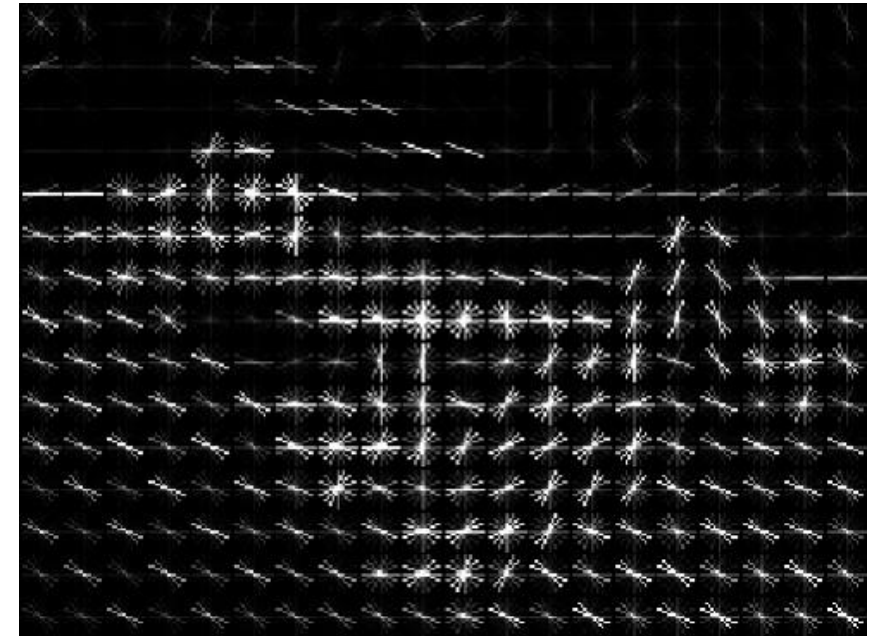
Weak edges

Strong diagonal edges



Edges in all directions

Captures texture and position, robust to small image changes



Example: 320x240 image gets divided into 40x30 bins; 8 directions per bin; feature vector has $30 \times 40 \times 9 = 10,800$ numbers

Image Features: Bag of Words (Data driven)

Step 1: Build codebook



Step 2: Encode images

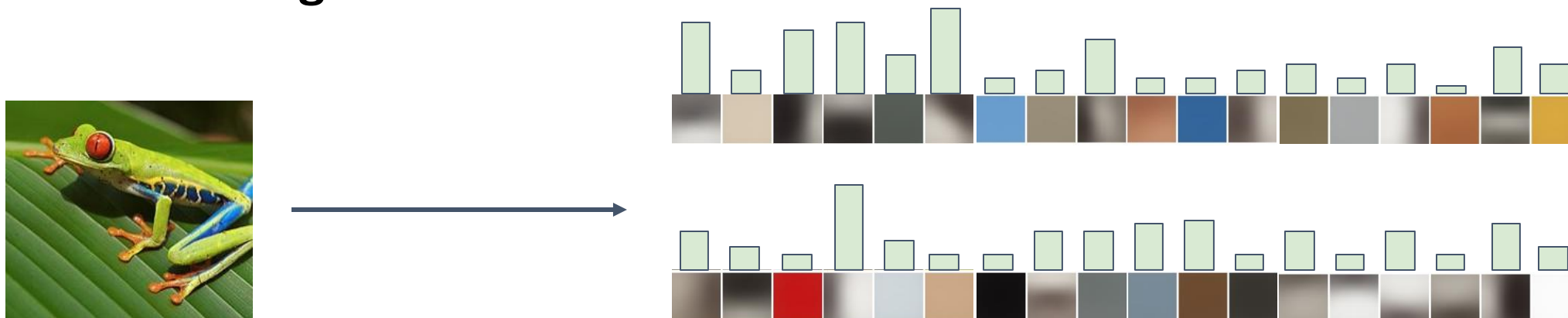
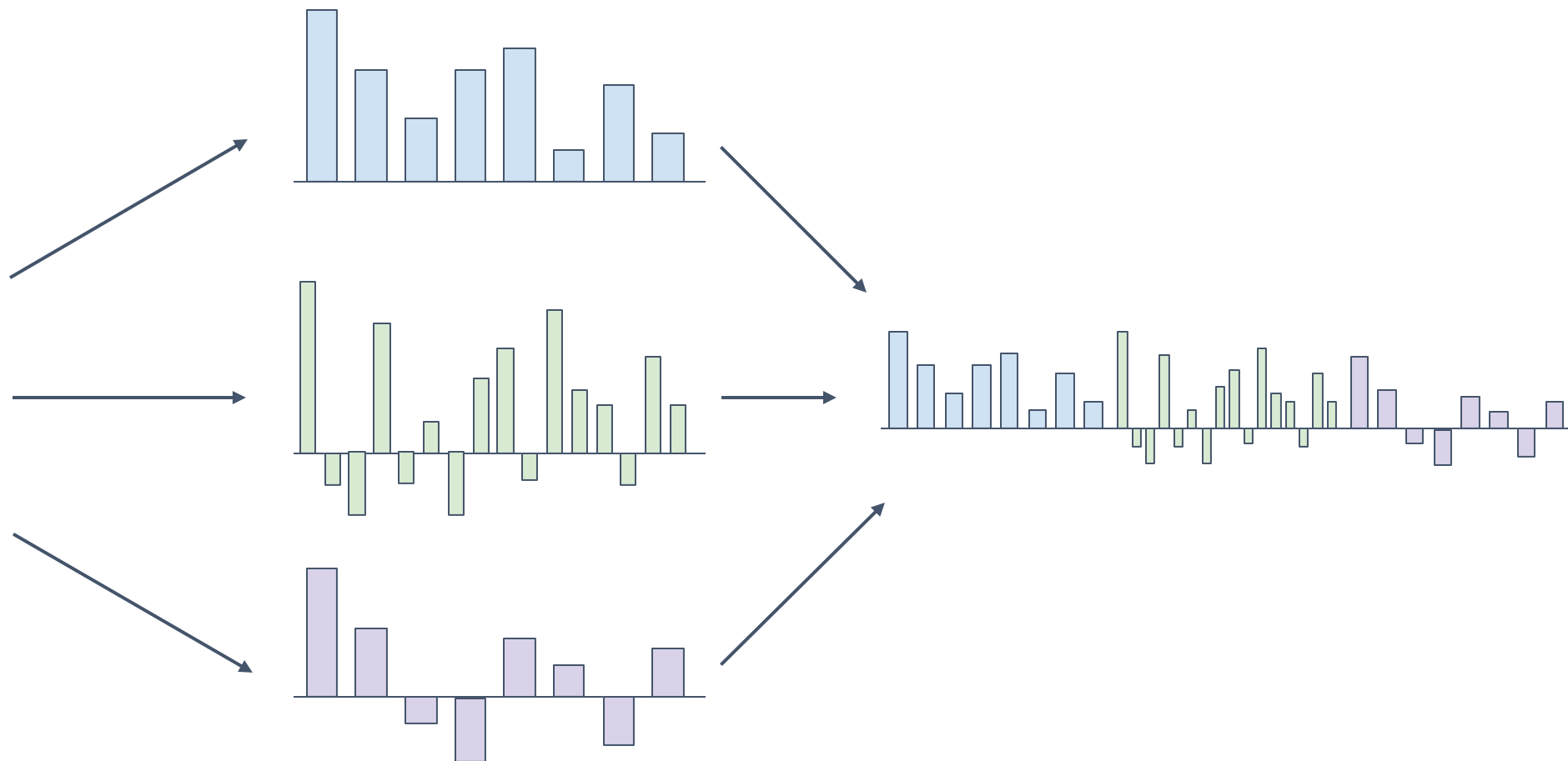


Image Features



Example: Winner of 2011 ImageNet challenge

Low-level feature extraction $\approx$ 10k patches per image

- SIFT: 128-dim
 - color: 96-dim
- } reduced to 64-dim with PCA

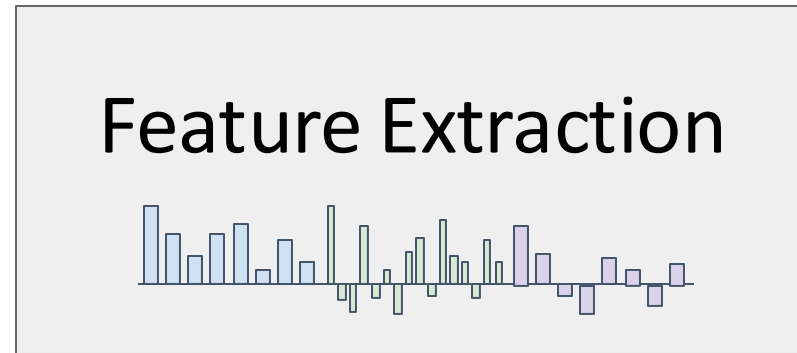
FV extraction and compression:

- $N=1,024$ Gaussians, $R=4$ regions $\Rightarrow$ 520K dim x 2
- compression: $G=8$, $b=1$ bit per dimension

One-vs-all SVM learning with SGD

Late fusion of SIFT and color systems

Image Features vs Neural Networks



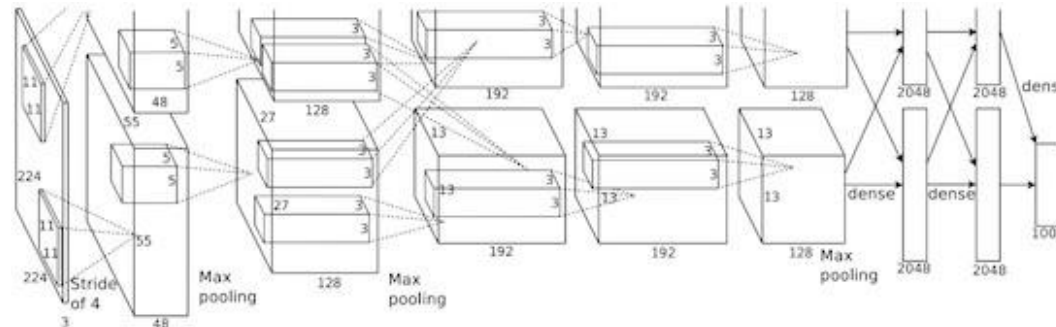
f



10 numbers giving scores for classes



training



Krizhevsky, Sutskever, and Hinton, "Imagenet classification with deep convolutional neural networks", NIPS 2012.
Figure copyright Krizhevsky, Sutskever, and Hinton, 2012.
Reproduced with permission.



10 numbers giving scores for classes

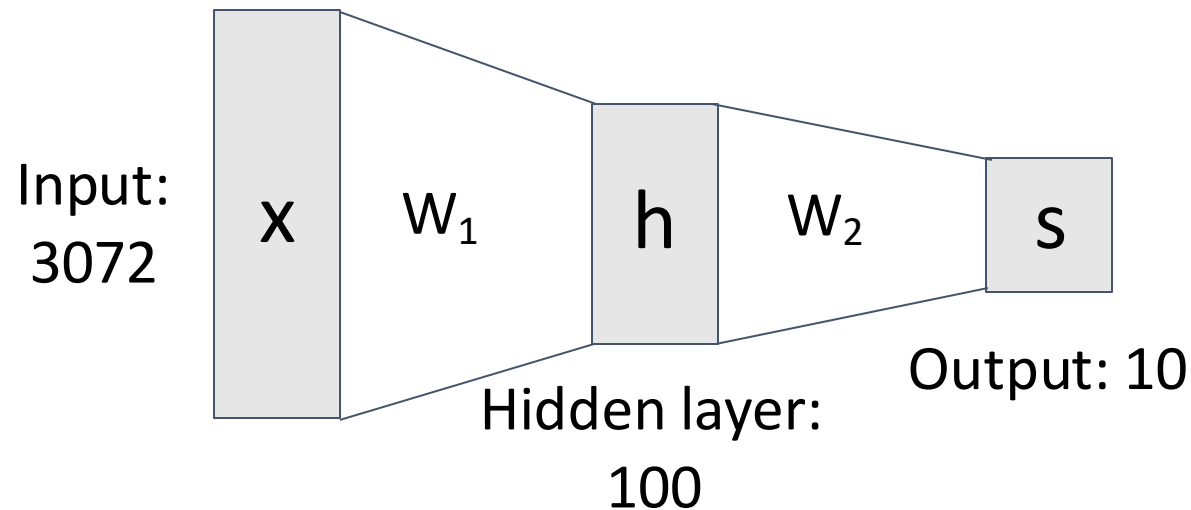


training



Multi-Layer Perceptron (MLP)

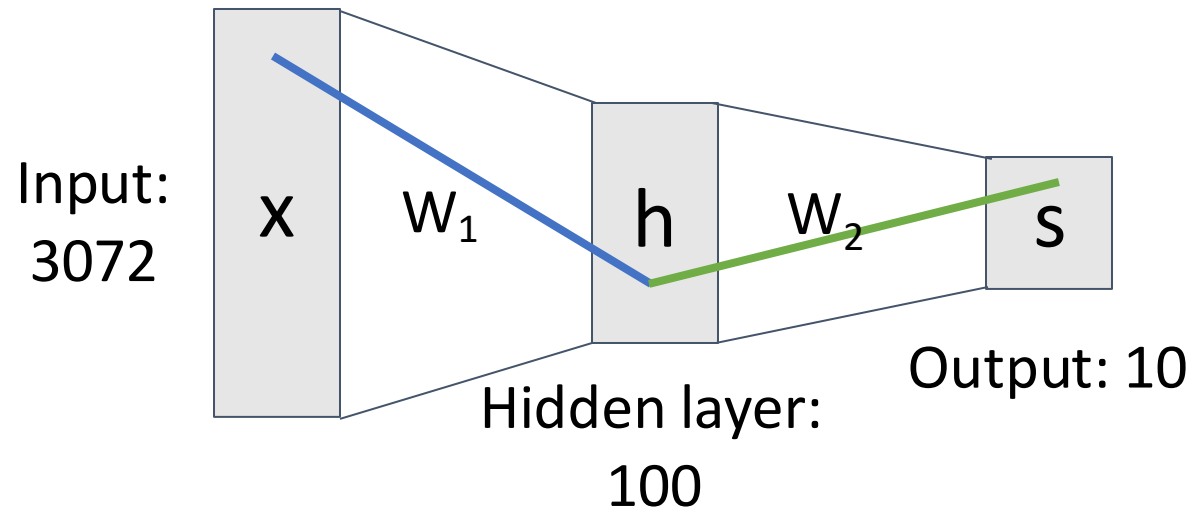
- **(Before)** Linear score function: $f = Wx$
- **(Now)** 2-layer Neural Network: $f = W_2 \max(0, W_1 x)$



$$x \in \mathbb{R}^D, W_1 \in \mathbb{R}^{H \times D}, W_2 \in \mathbb{R}^{C \times H}$$

Multi-Layer Perceptron (MLP)

Element (i, j)
of W_1 gives
the effect on
 h_i from x_j

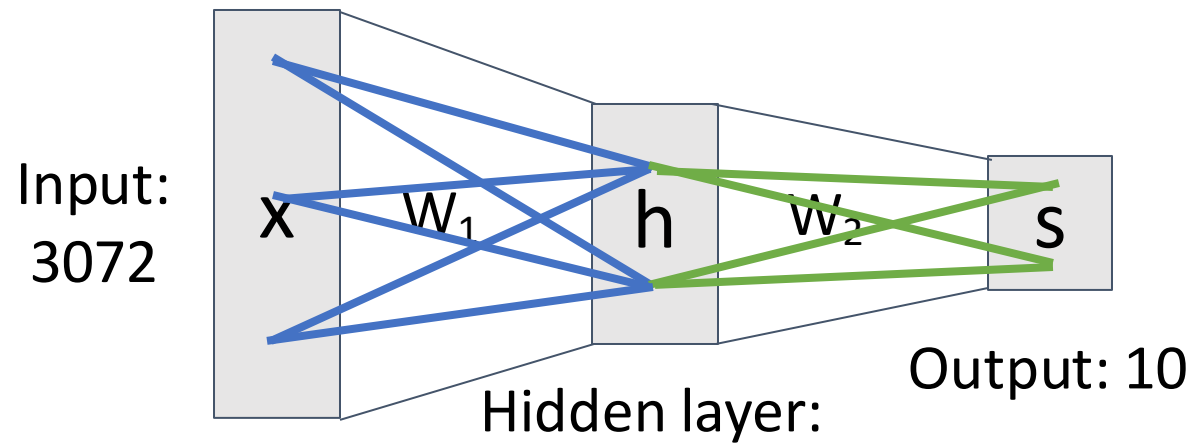


Element (i, j)
of W_2 gives
the effect on
 s_i from h_j

$$x \in \mathbb{R}^D, W_1 \in \mathbb{R}^{H \times D}, W_2 \in \mathbb{R}^{C \times H}$$

Also known as “Fully-connected” NN

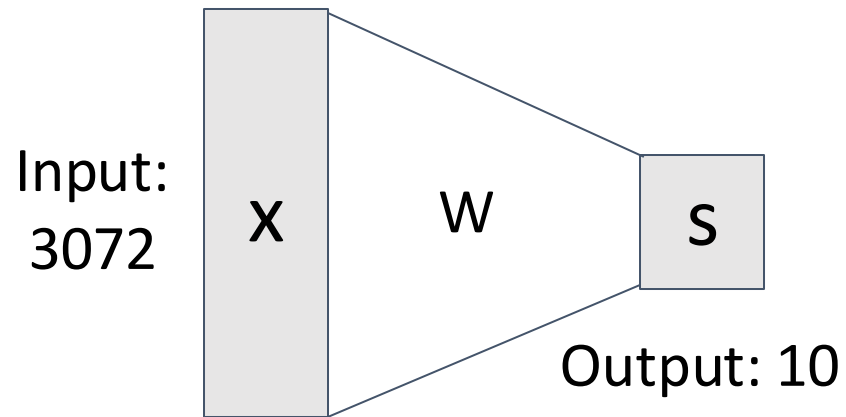
All elements
of x affect all
elements of h



All elements
of h affect all
elements of s

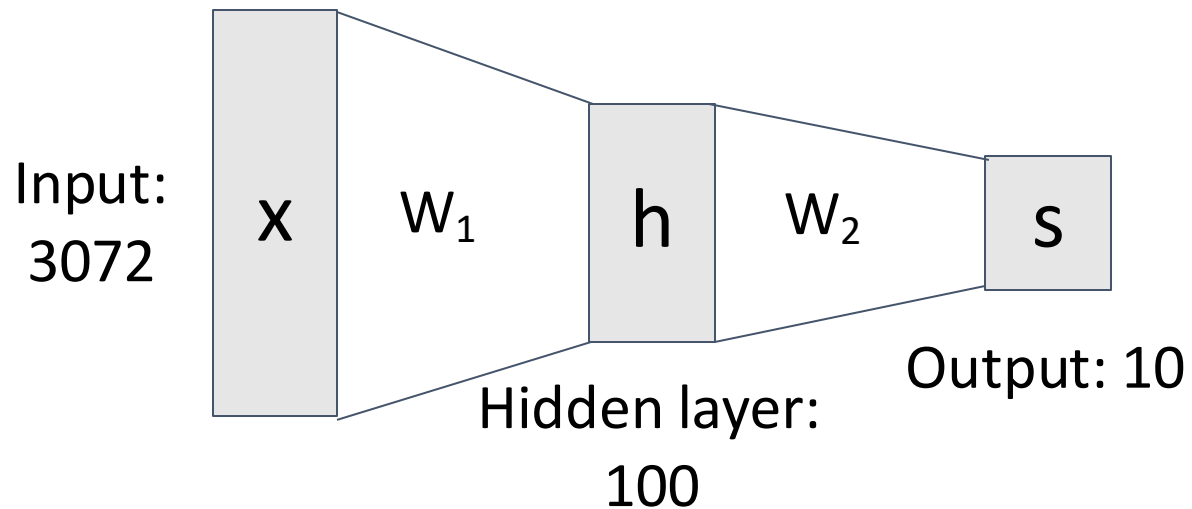
Reminder: linear classifier

- Linear classifier: One template per class



Now: 2-layer Neural Network

- First layer is bank of templates
- Second layer re-combines templates

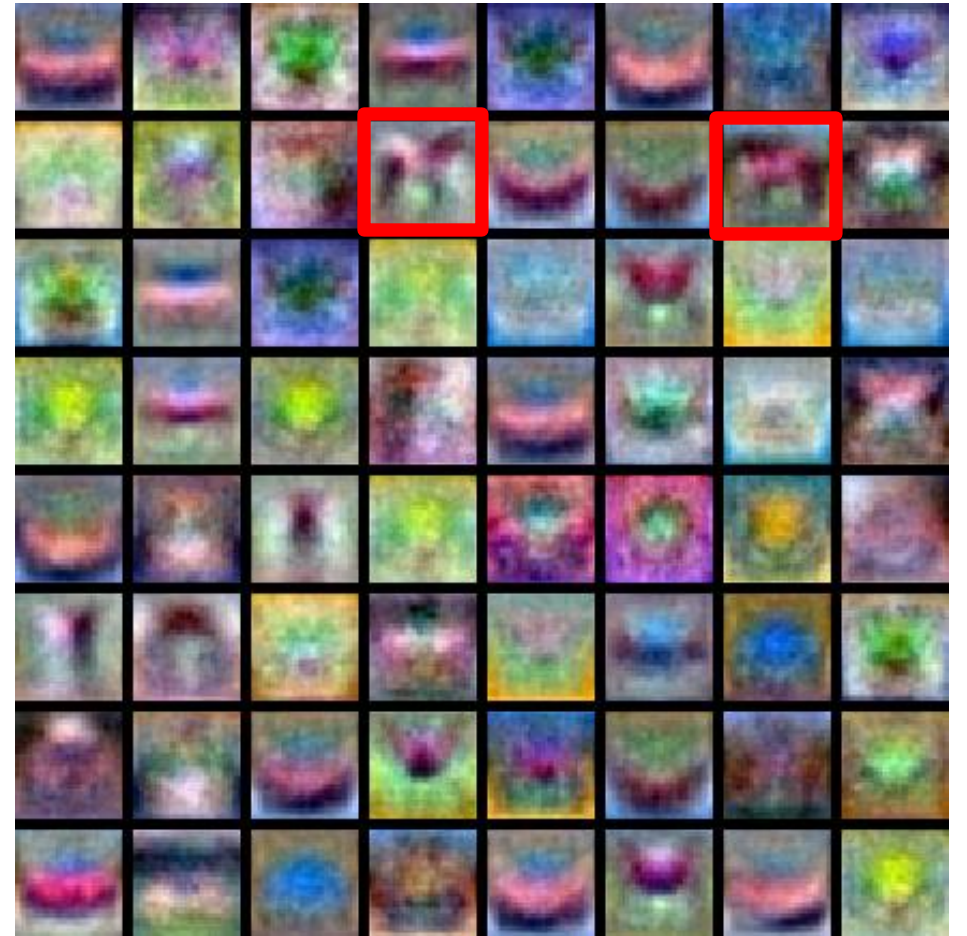


$$x \in \mathbb{R}^D, W_1 \in \mathbb{R}^{H \times D}, W_2 \in \mathbb{R}^{C \times H}$$



2-layer Neural Network

- Can use different templates to cover multiple modes of a class!

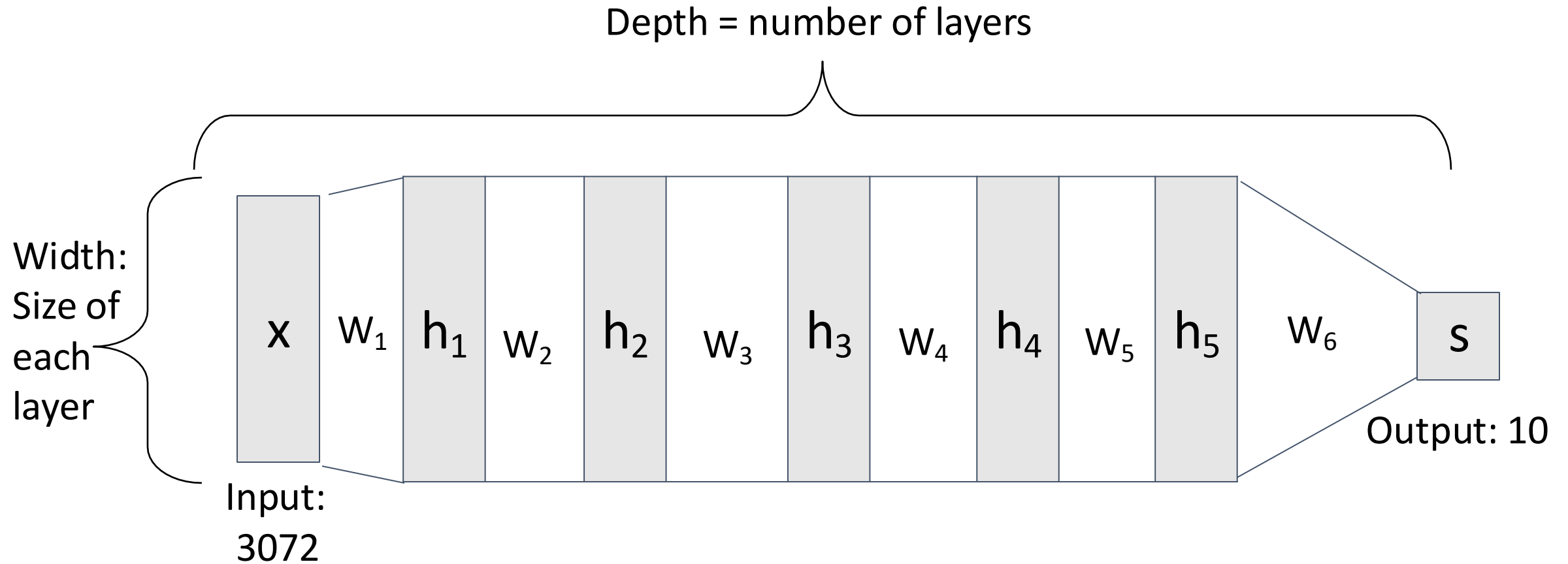


2-layer Neural Network

- “Distributed representation”: Most templates not interpretable



Deep Neural Networks



$$s = W_6 \max(0, W_6 \max(0, W_5 \max(0, W_4 \max(0, W_3 \max(0, W_2 \max(0, W_1 x))))))$$

Activation Functions

- This is called the **activation function** of the neural network

$$f = W_2 \max(0, W_1 x)$$

- **Q:** What happens if we build a neural network with no activation function?

$$s = W_2 W_1 x$$

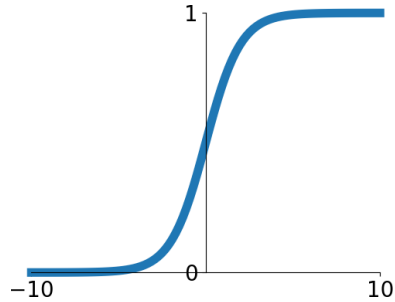
$$W_3 = W_2 W_1 \in \mathbb{R}^{C \times H} \quad s = W_3 x$$

- **A:** We end up with a linear classifier!

Activation Functions

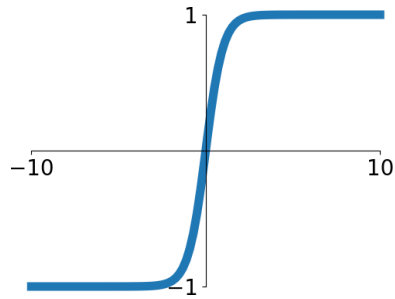
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



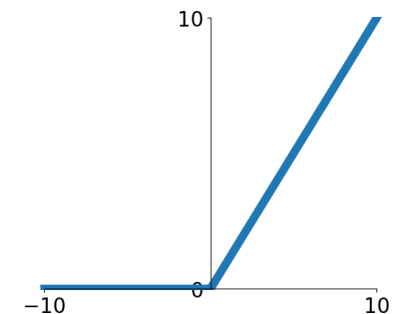
tanh

$$\tanh(x)$$



ReLU

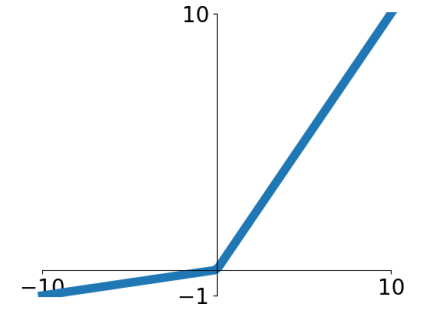
$$\max(0, x)$$



ReLU is a good default choice
for most problems

Leaky ReLU

$$\max(0.1x, x)$$

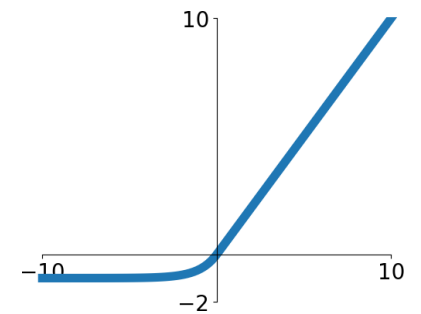


Maxout

$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

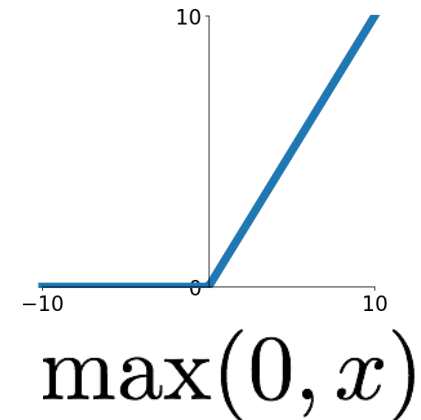
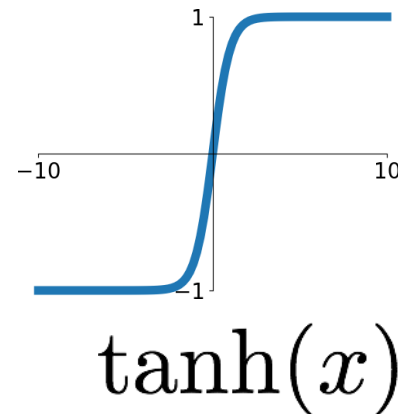
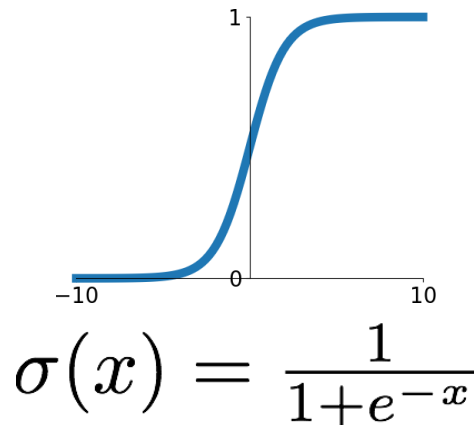
ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$



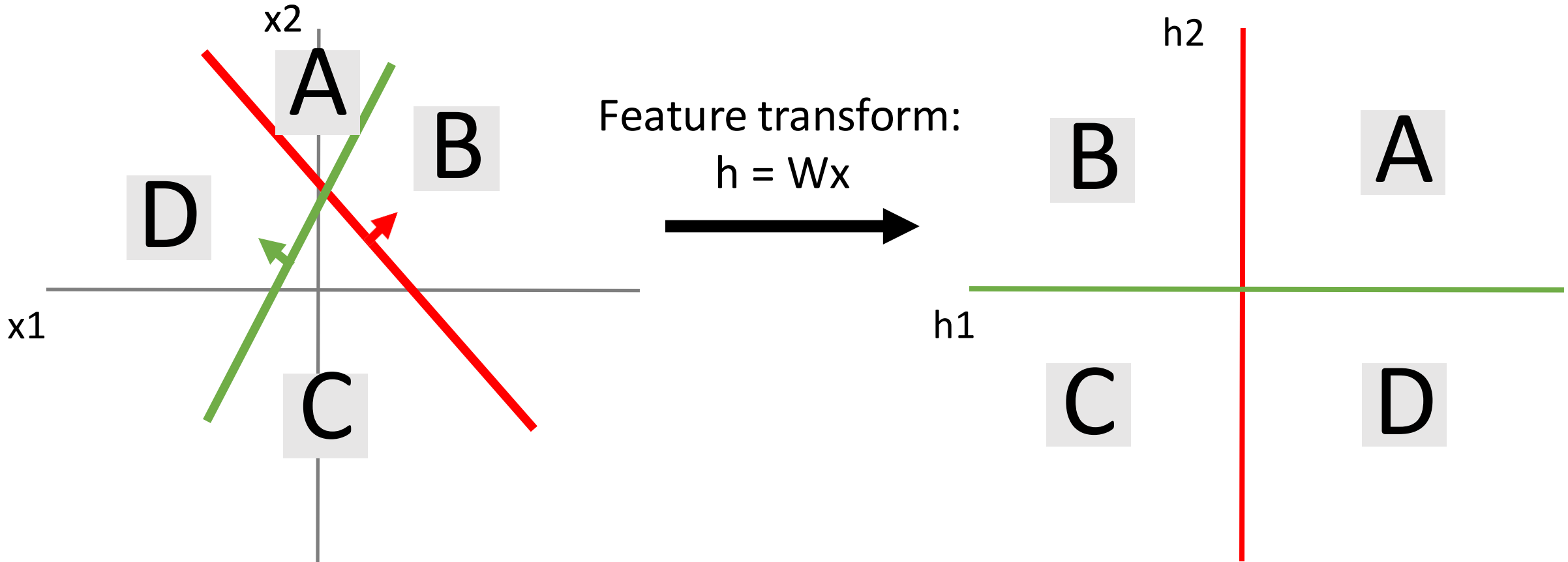
Why ReLU is a good.

- **Fast.** avoiding expensive operations (e.g., exponentials)
- Sparse activations
 - Fewer active neurons; lower computation and memory use.
 - (Mimics brain-like sparse neural activity)
- **Gradient flow:** Mitigates vanishing gradient issues common in sigmoid/tanh

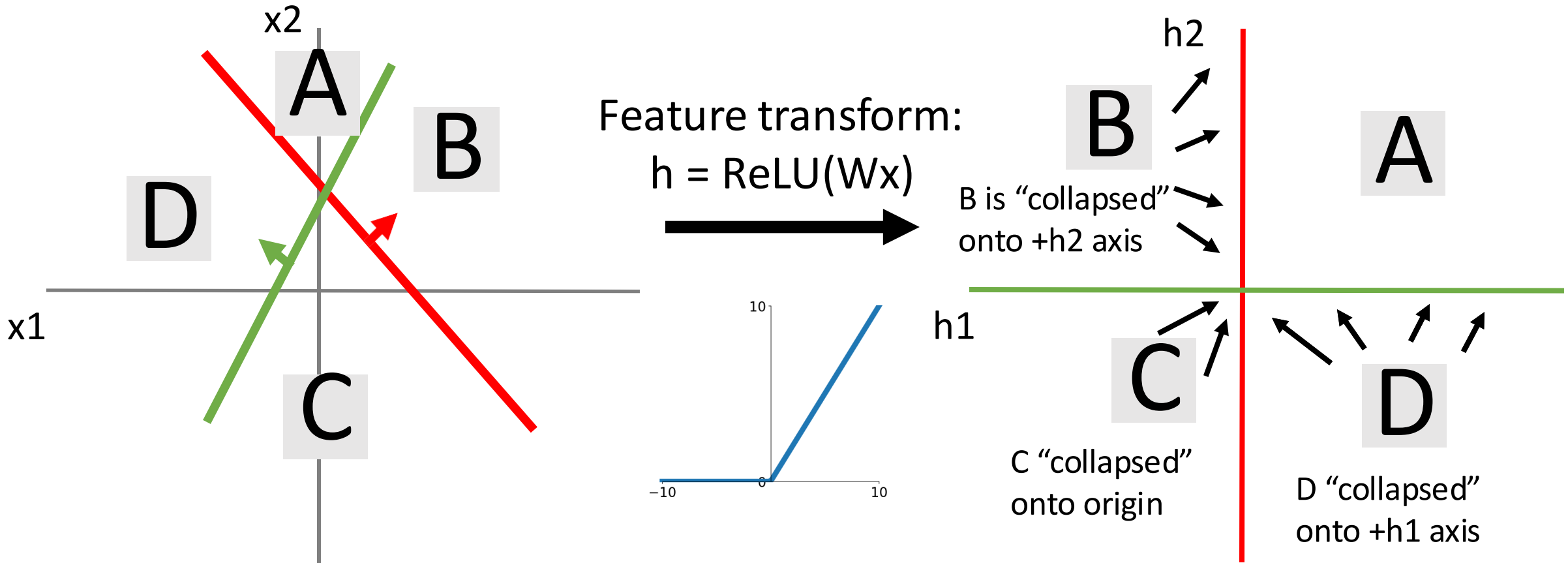


Multi-layer can also be seen as space warping

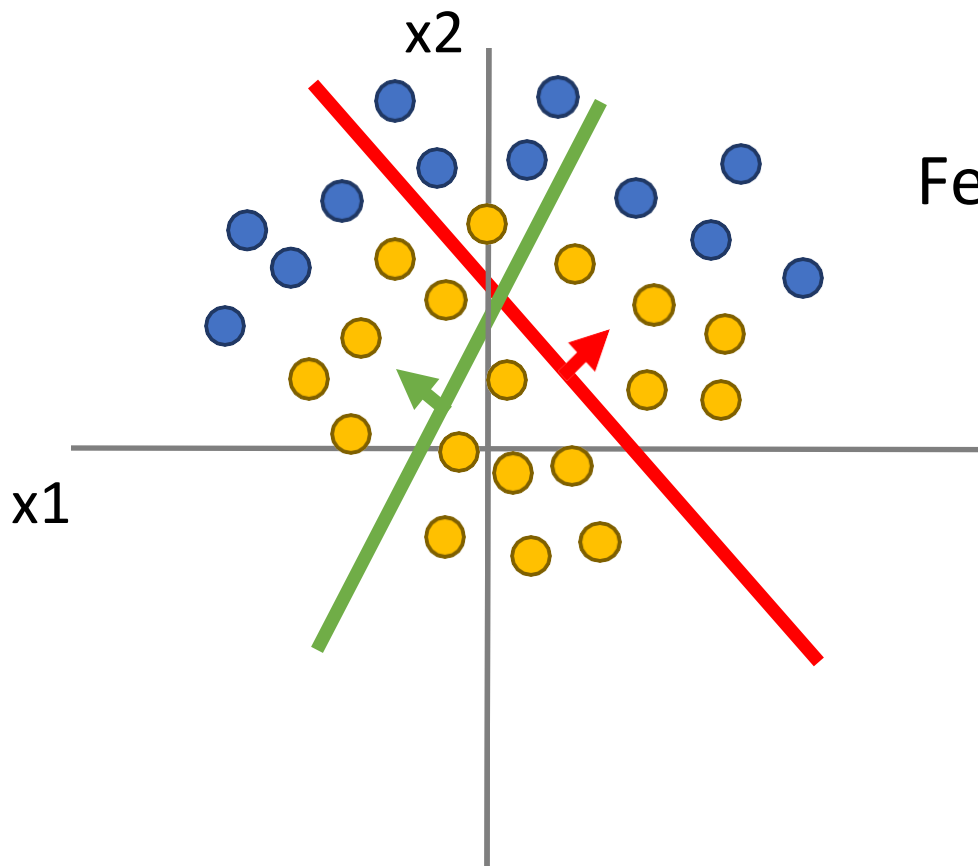
Consider a linear transform: $h = Wx$
Where x , h are both 2-dimensional



Space Warping w/ ReLU



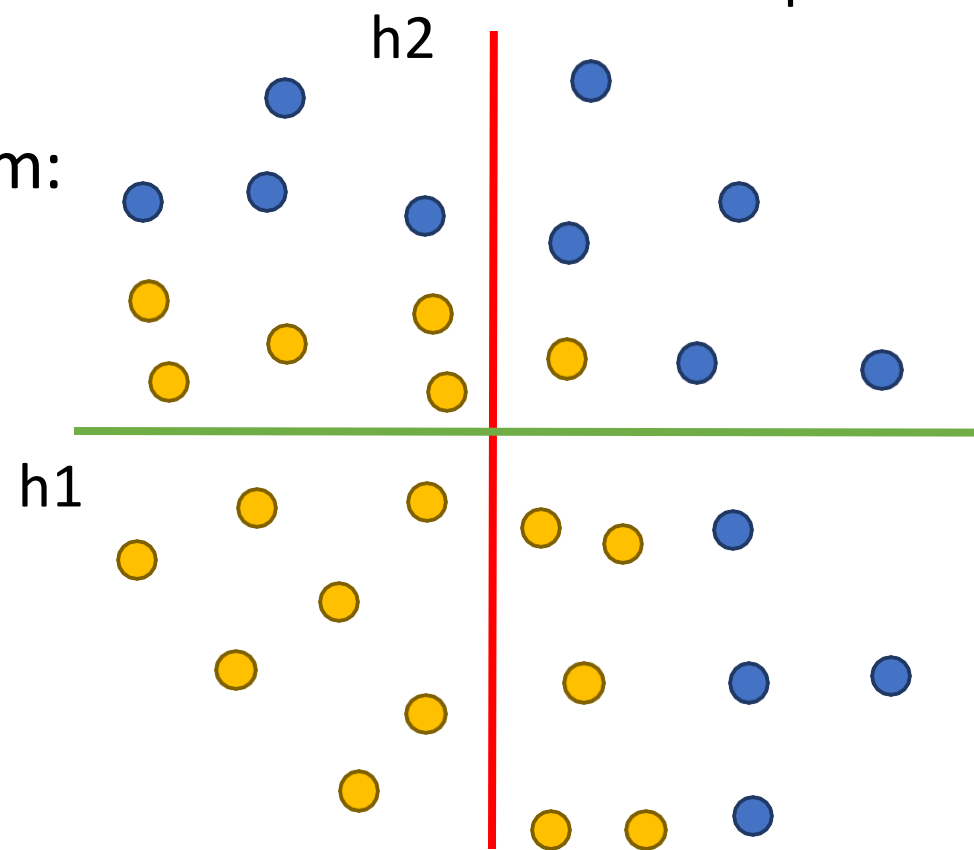
Points not linearly separable in original space



Feature transform:
 $h = Wx$

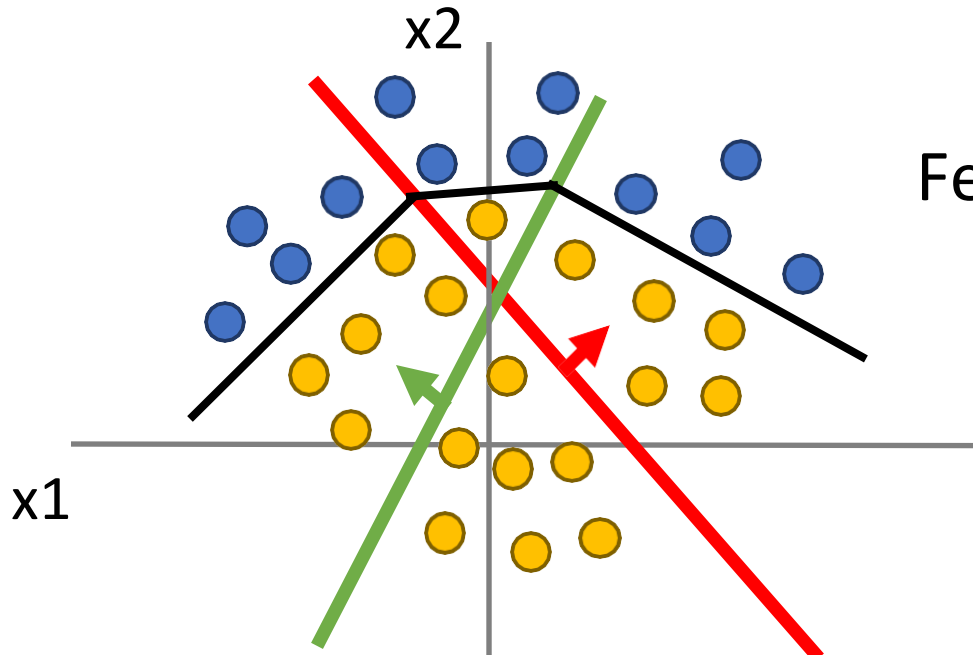


Not linearly separable in feature space



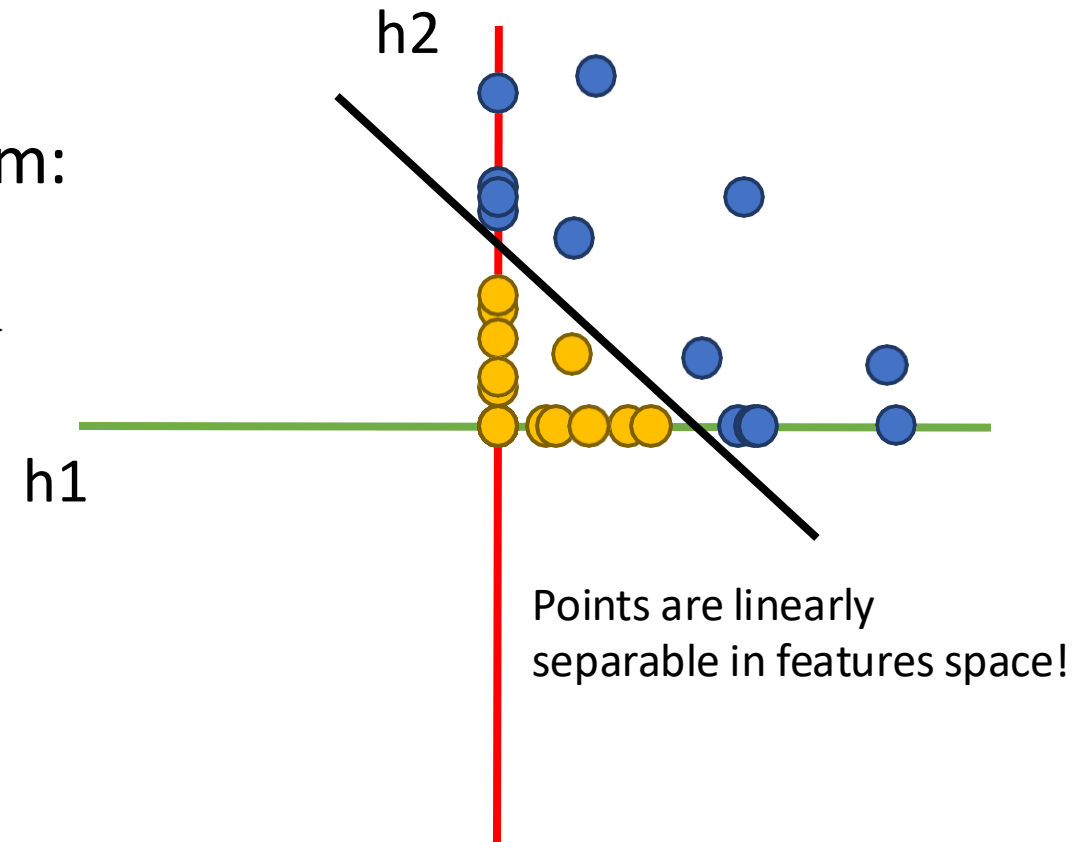
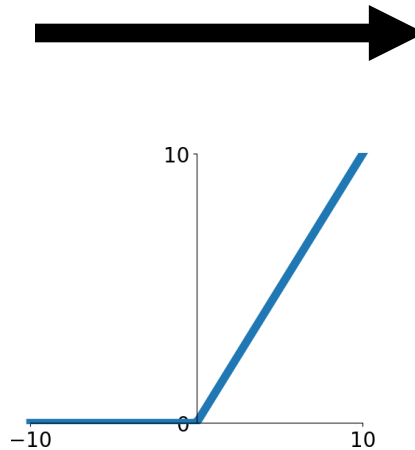
Nonlinear boundary

Points not linearly separable in original space



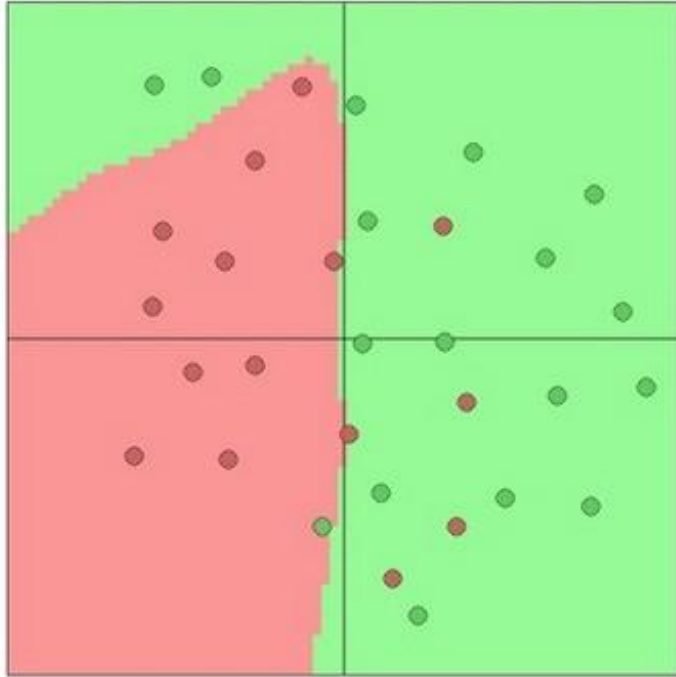
Linear classifier in feature space gives nonlinear classifier in original space

Feature transform:
 $h = \text{ReLU}(Wx)$

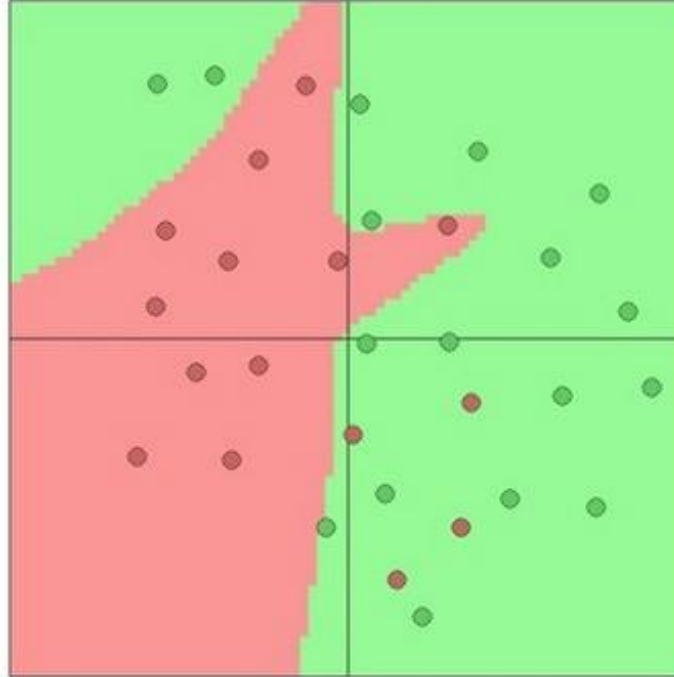


Setting the number of layers and their sizes

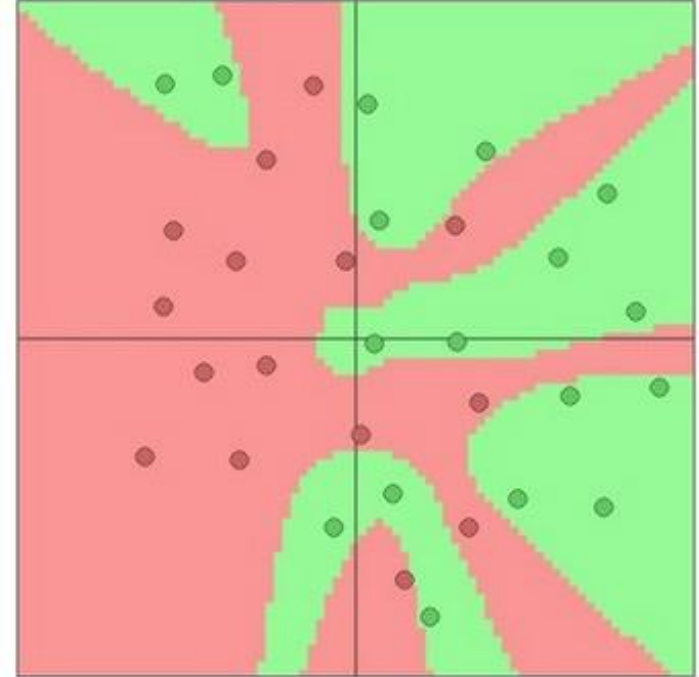
3 hidden units



6 hidden units



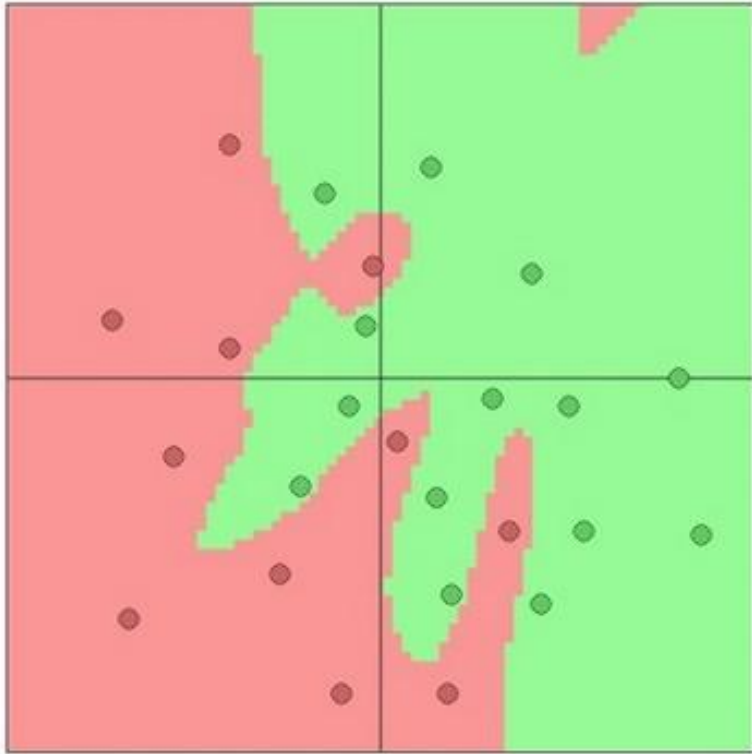
20 hidden units



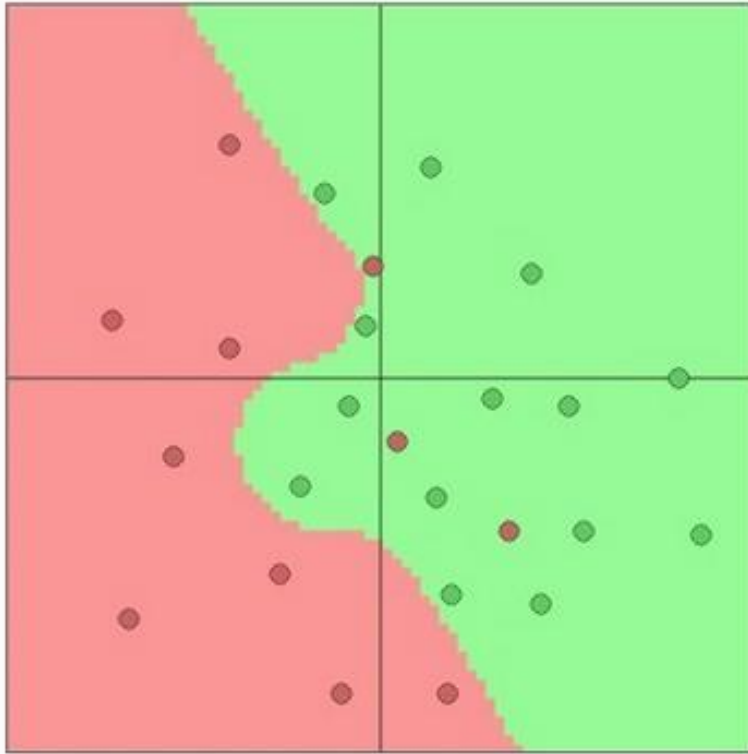
More hidden units = more capacity

Remember to use regularization (e.g. L2)

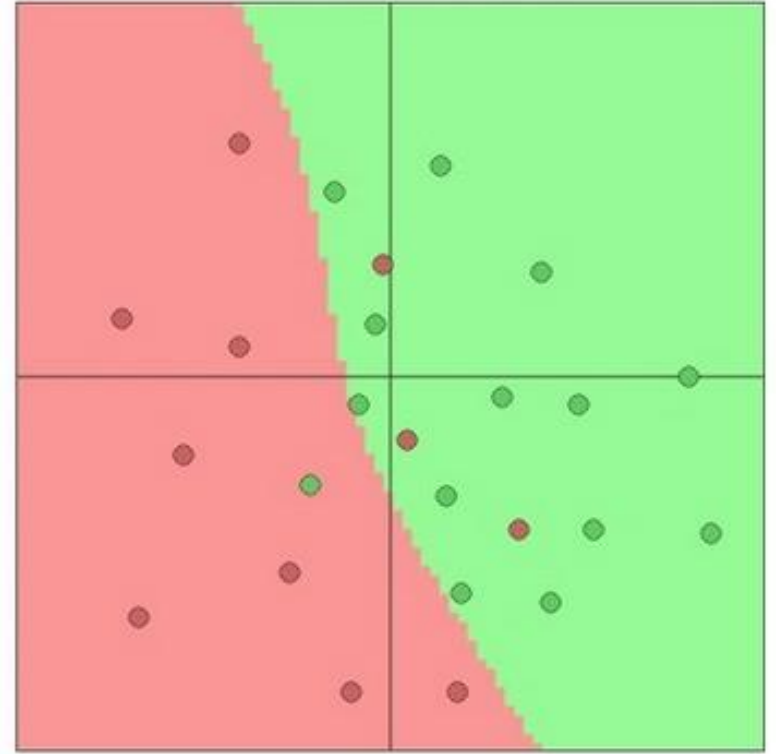
$\lambda = 0.001$



$\lambda = 0.01$



$\lambda = 0.1$



Problem: How to compute gradients?

$$s = f(x; W_1, W_2) = W_2 \max(0, W_1 x) \quad \text{Nonlinear score function}$$

$$L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1) \quad \text{SVM Loss on predictions}$$

$$R(W) = \sum_k W_k^2 \quad \text{Regularization}$$

$$L = \frac{1}{N} \sum_{i=1}^N L_i + \lambda R(W_1) + \lambda R(W_2) \quad \text{Total loss: data loss + regularization}$$

If we can compute $\frac{\partial L}{\partial W_1}, \frac{\partial L}{\partial W_2}$ then we can learn W_1 and W_2

(Bad) Idea: Derive $\nabla_W L$ on paper

$$s = f(x; W) = Wx$$

$$L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1)$$

$$= \sum_{j \neq y_i} \max(0, W_{j,:} \cdot x + W_{y_i,:} \cdot x + 1)$$

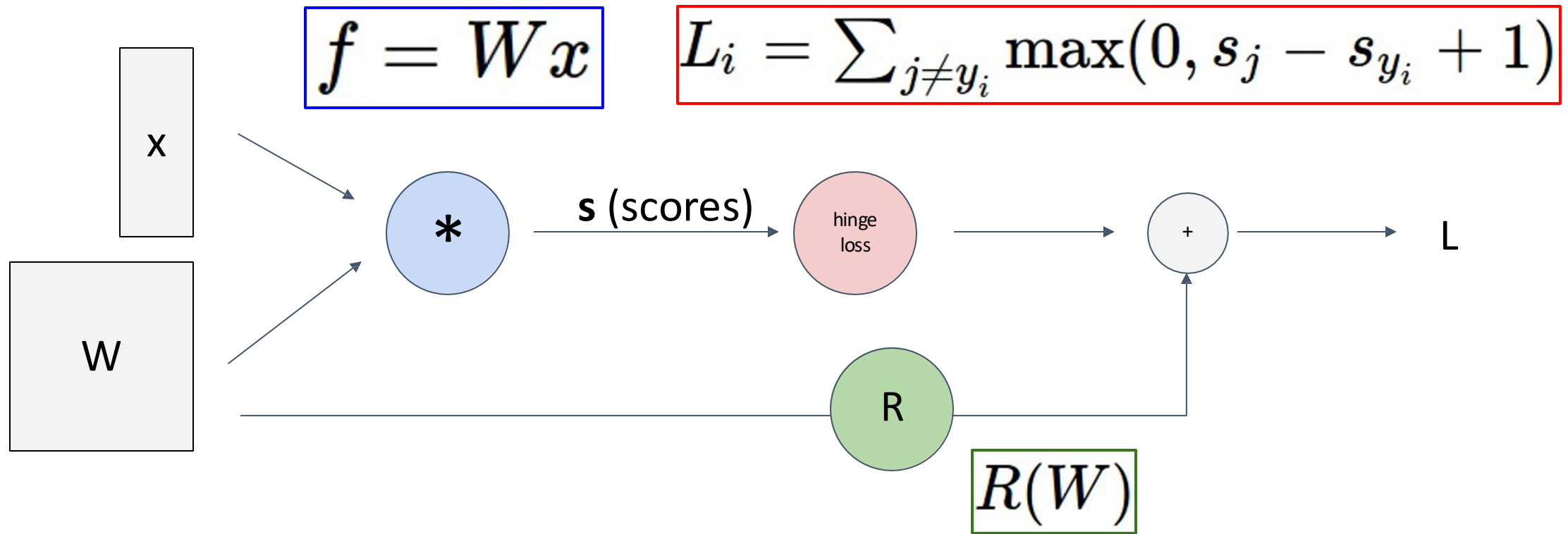
$$L = \frac{1}{N} \sum_{i=1}^N L_i + \lambda \sum_k W_k^2$$

$$= \frac{1}{N} \sum_{i=1}^N \sum_{j \neq y_i} \max(0, W_{j,:} \cdot x + W_{y_i,:} \cdot x + 1) + \lambda \sum_k W_k^2$$

$$\nabla_W L = \nabla_W \left(\frac{1}{N} \sum_{i=1}^N \sum_{j \neq y_i} \max(0, W_{j,:} \cdot x + W_{y_i,:} \cdot x + 1) + \lambda \sum_k W_k^2 \right)$$

tedious, not modular, not feasible
for very complex models.

Backpropagation Graphs



Forward pass computes outputs

Backward pass computes gradients

Backpropagation: Simple Example

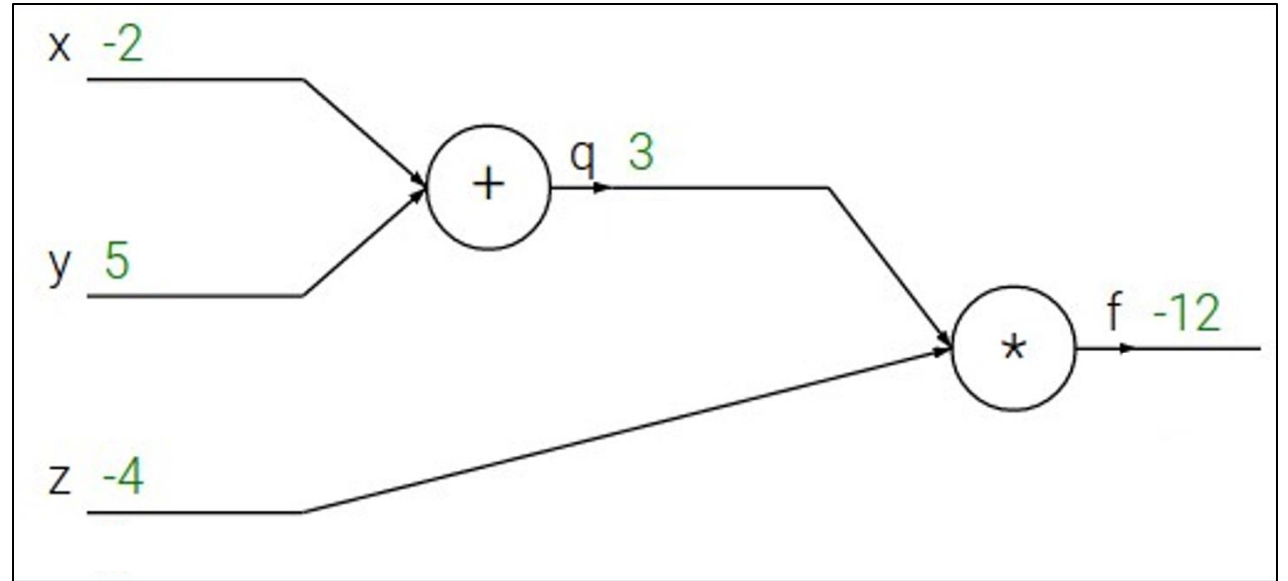
- **1. Forward pass:** Compute outputs

$$q = x + y$$

$$f = qz$$

- **Backward pass:** Compute derivatives

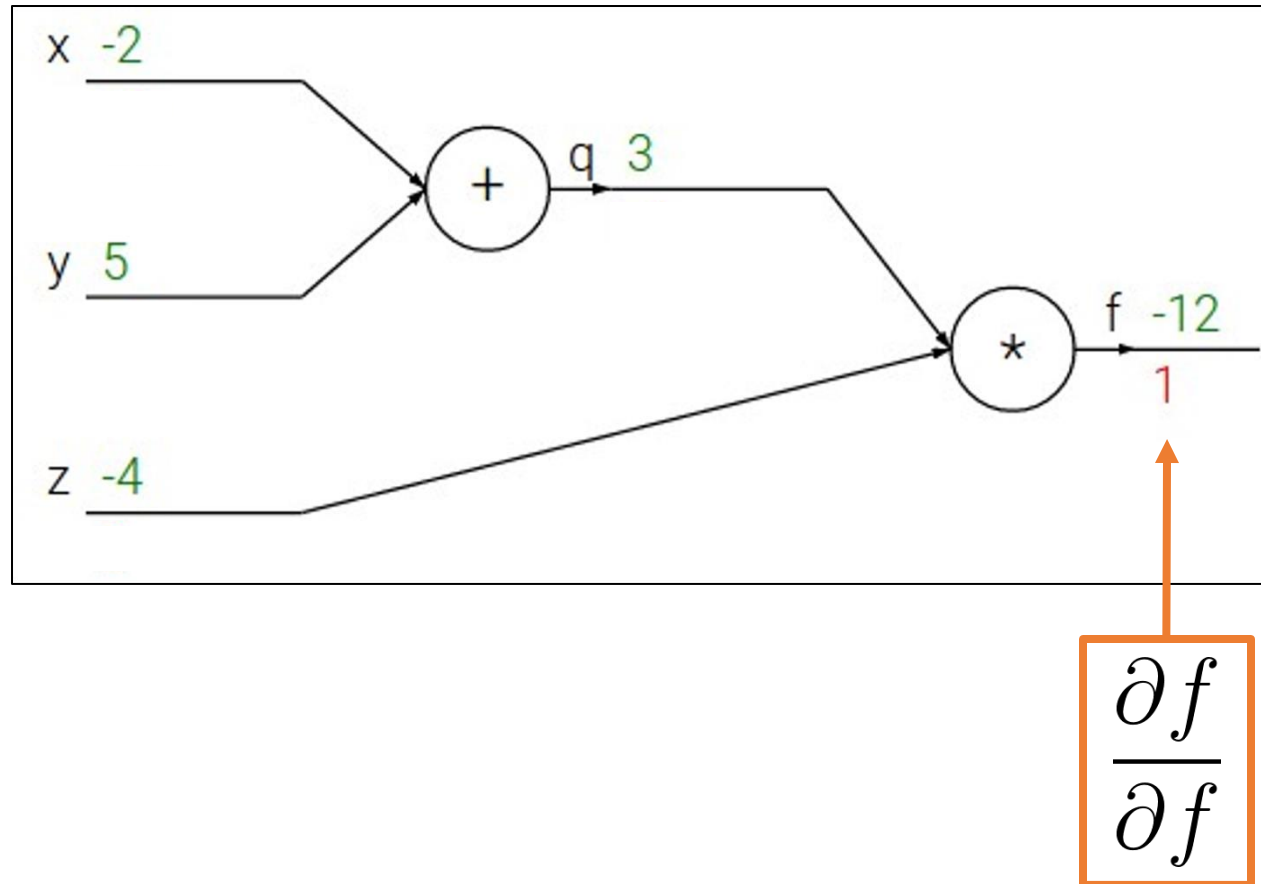
$$\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$$



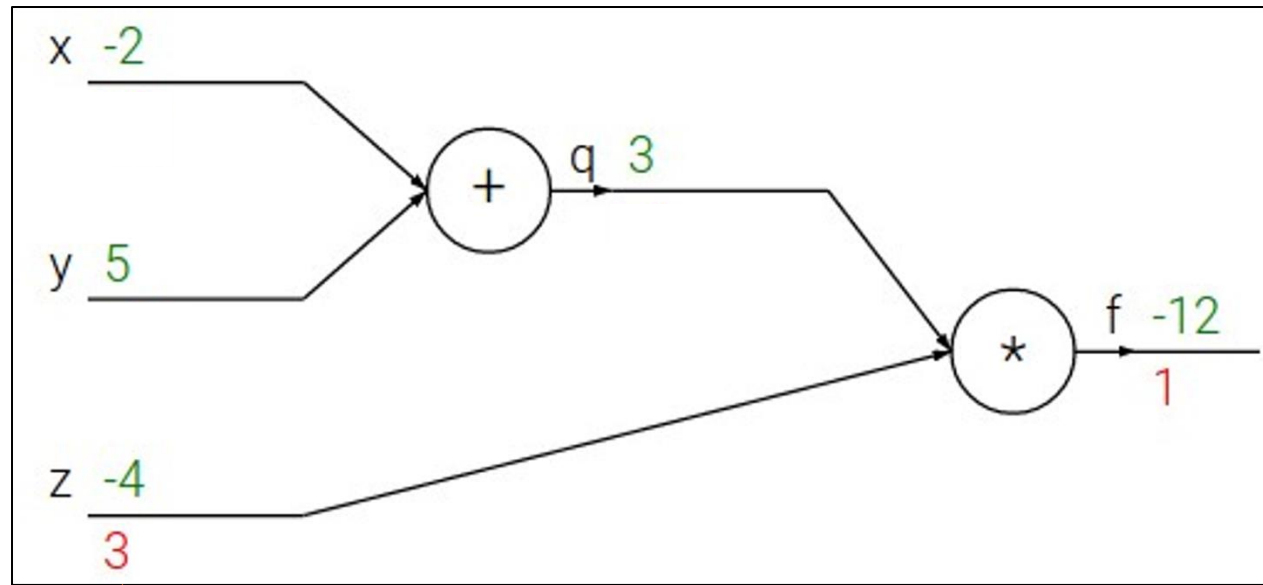
$$f(x, y, z) = (x + y)z$$

$$\text{e.g. } x = -2, y = 5, z = -4$$

Backpropagation



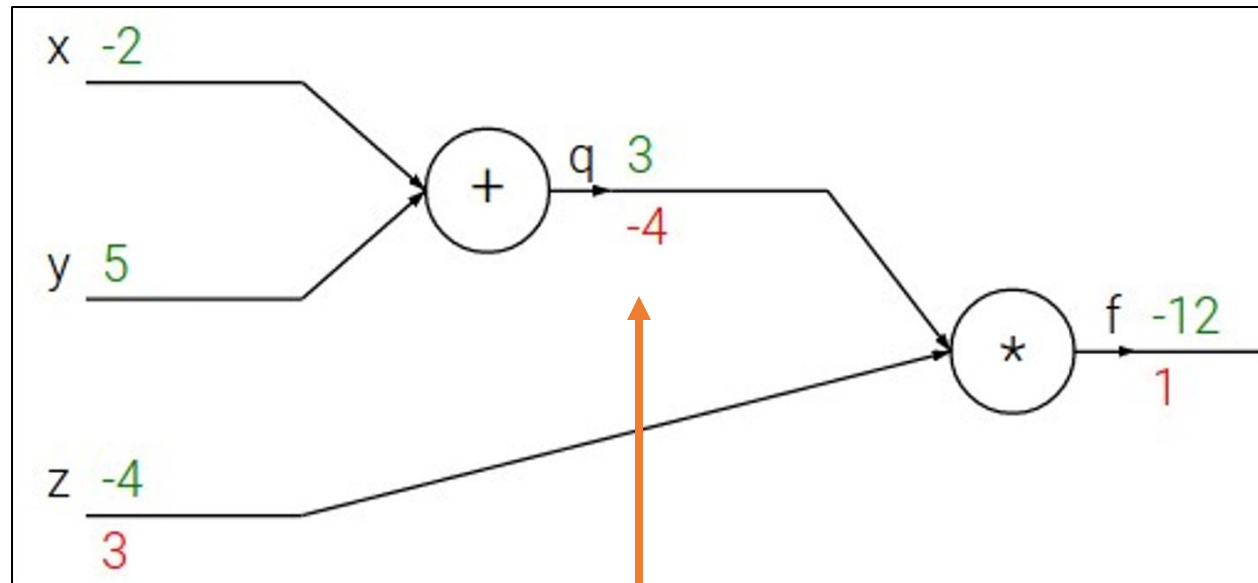
Backpropagation



$$f = qz$$

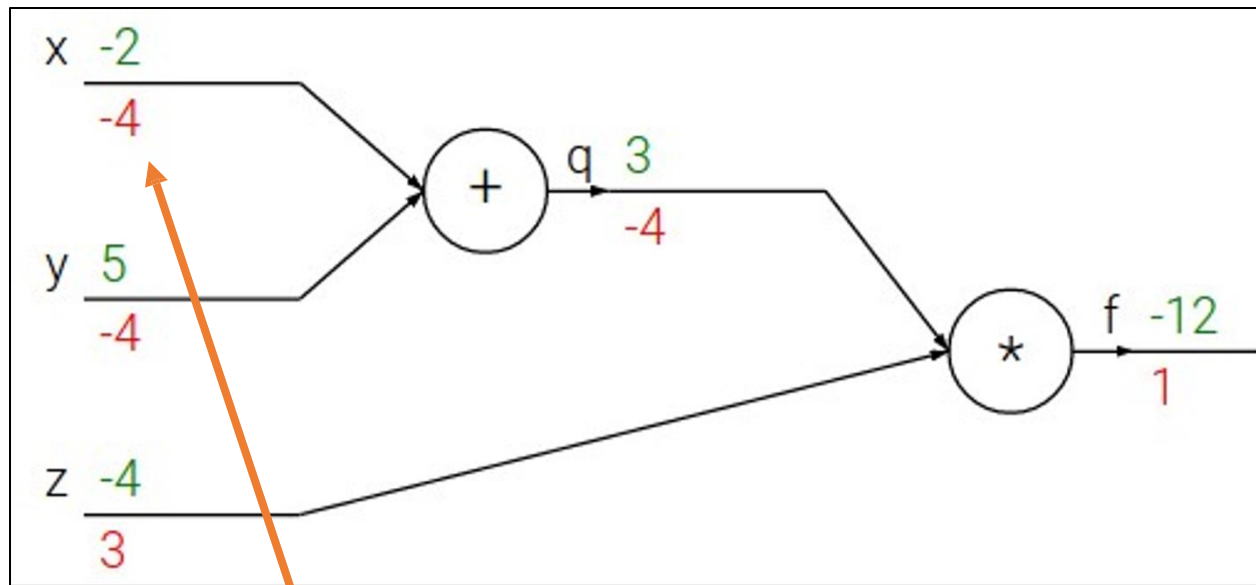
$$\frac{\partial f}{\partial z} = q$$

Backpropagation



$$f = qz$$

$$\frac{\partial f}{\partial q} = z$$



Chain Rule

$$q = x + y$$

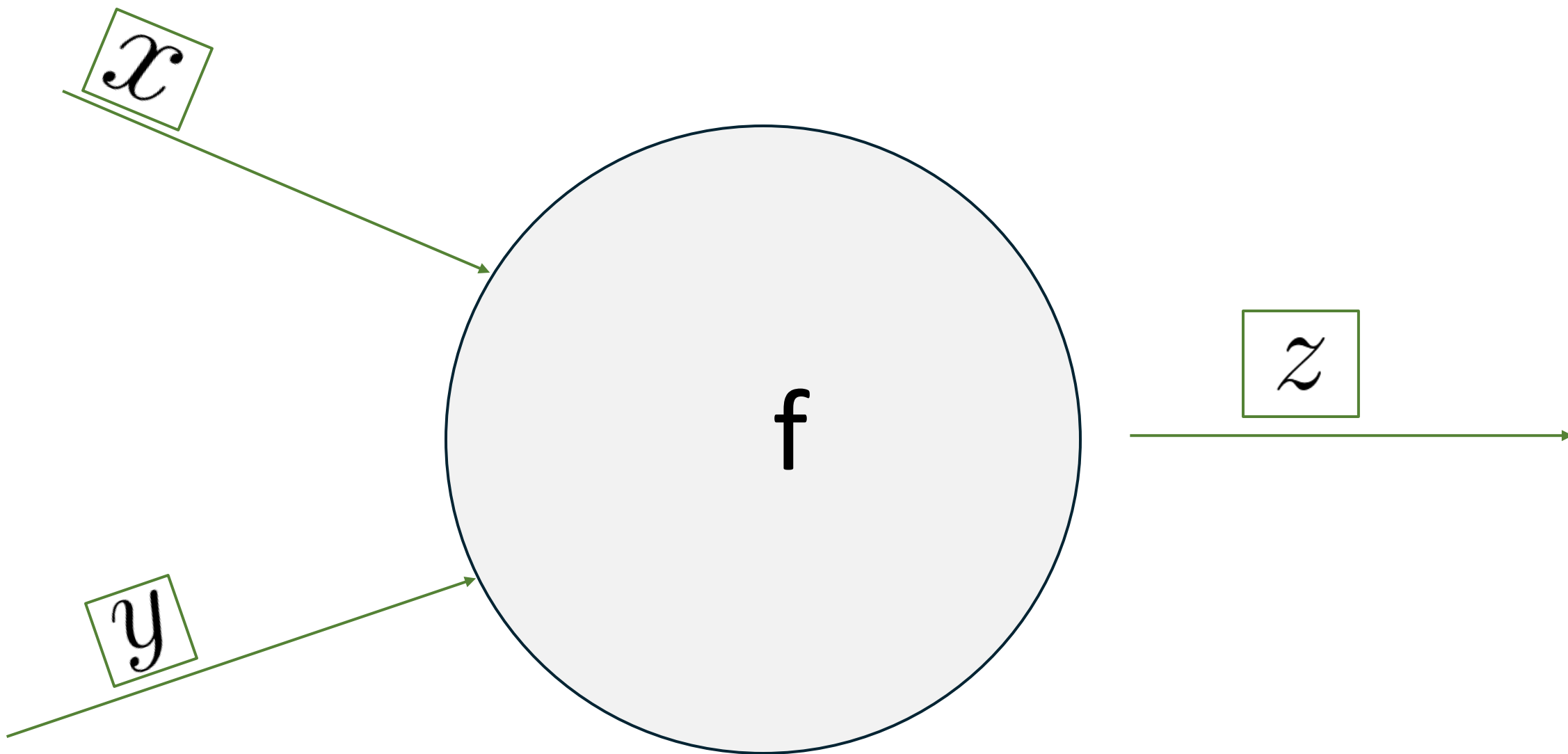
$$\frac{\partial f}{\partial x} = \frac{\partial q}{\partial x} \frac{\partial f}{\partial q}$$

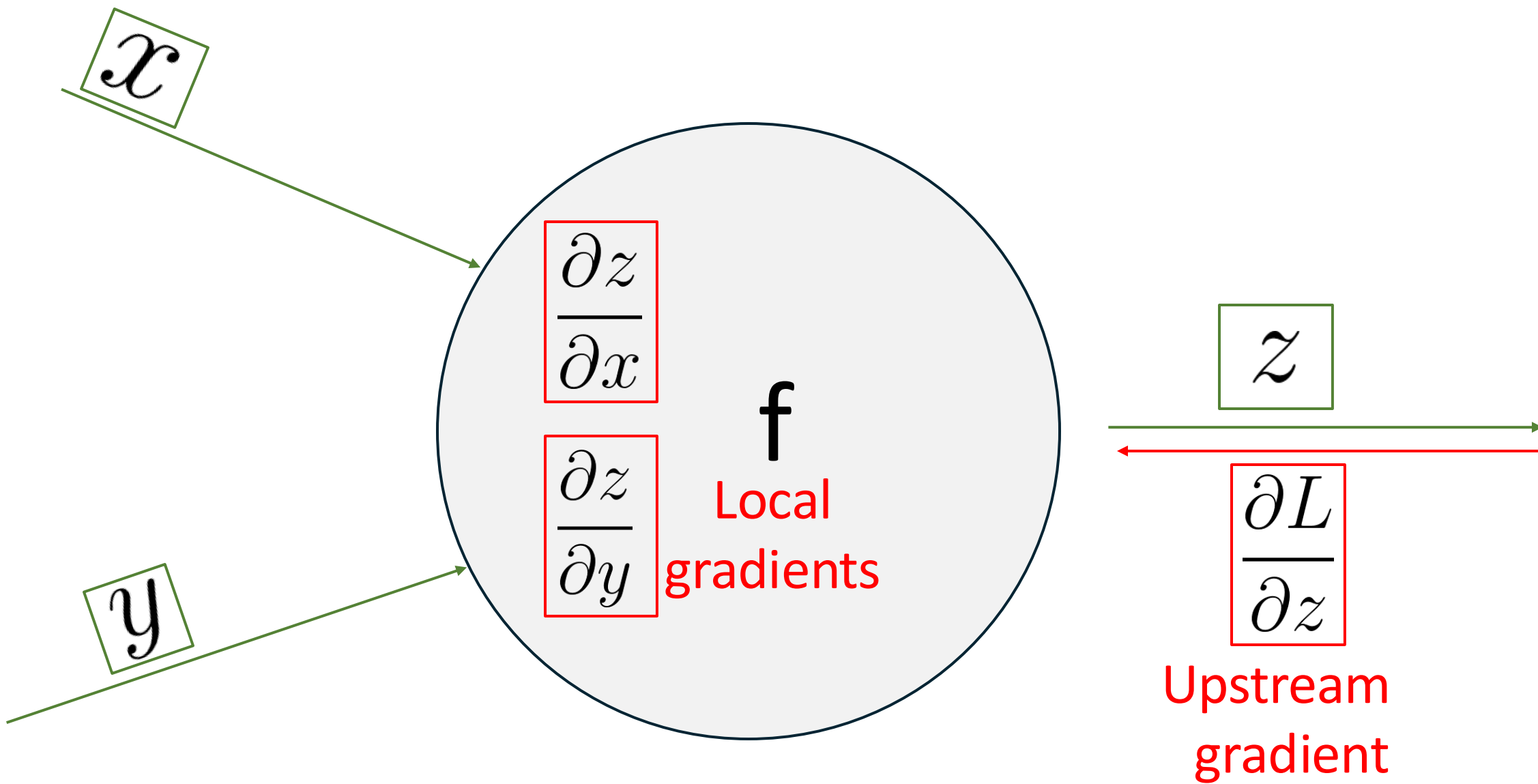
$$\frac{\partial q}{\partial x} = 1$$

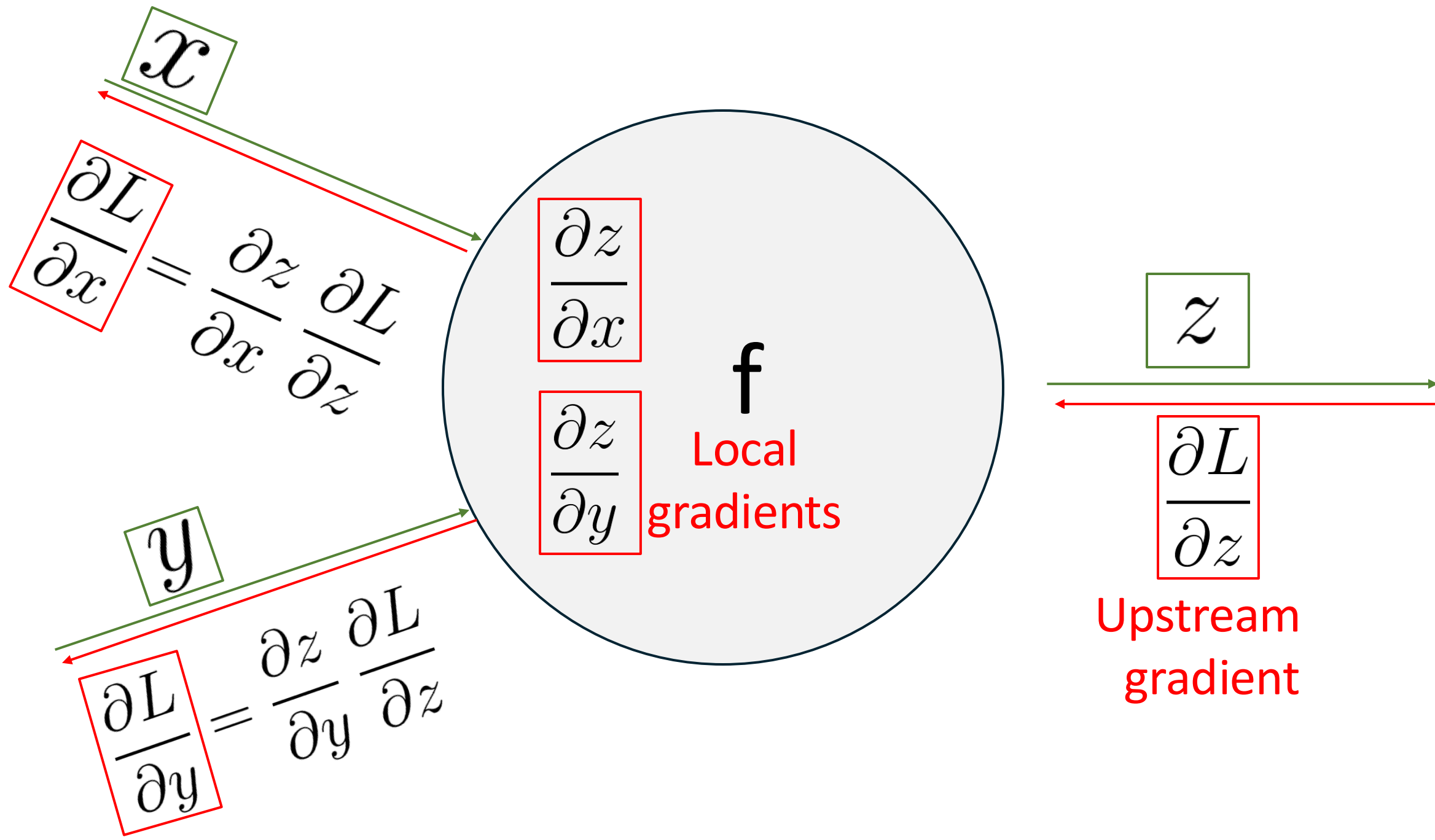
**Downstream
Gradient**

**Local
Gradient**

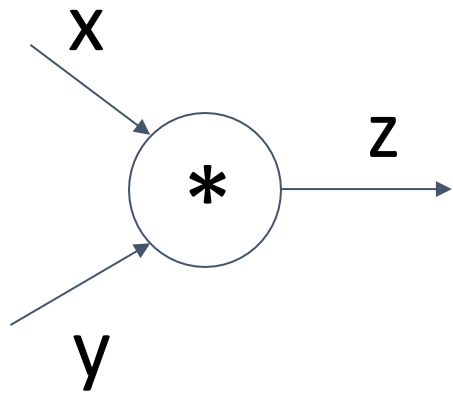
**Upstream
Gradient**







Example: PyTorch Autograd Functions



(x,y,z are scalars)

```
class Multiply(torch.autograd.Function):  
    @staticmethod  
    def forward(ctx, x, y):  
        ctx.save_for_backward(x, y)  
        z = x * y  
        return z  
    @staticmethod  
    def backward(ctx, grad_z):  
        x, y = ctx.saved_tensors  
        grad_x = y * grad_z    # dz/dx * dL/dz  
        grad_y = x * grad_z    # dz/dy * dL/dz  
        return grad_x, grad_y
```

Need to stash some
values for use in
backward

Upstream
gradient

Multiply upstream
and local gradients

Recap: Vector Derivatives

$$x \in \mathbb{R}, y \in \mathbb{R}$$

Regular derivative:

$$\frac{\partial y}{\partial x} \in \mathbb{R}$$

If x changes by a small amount, how much will y change?

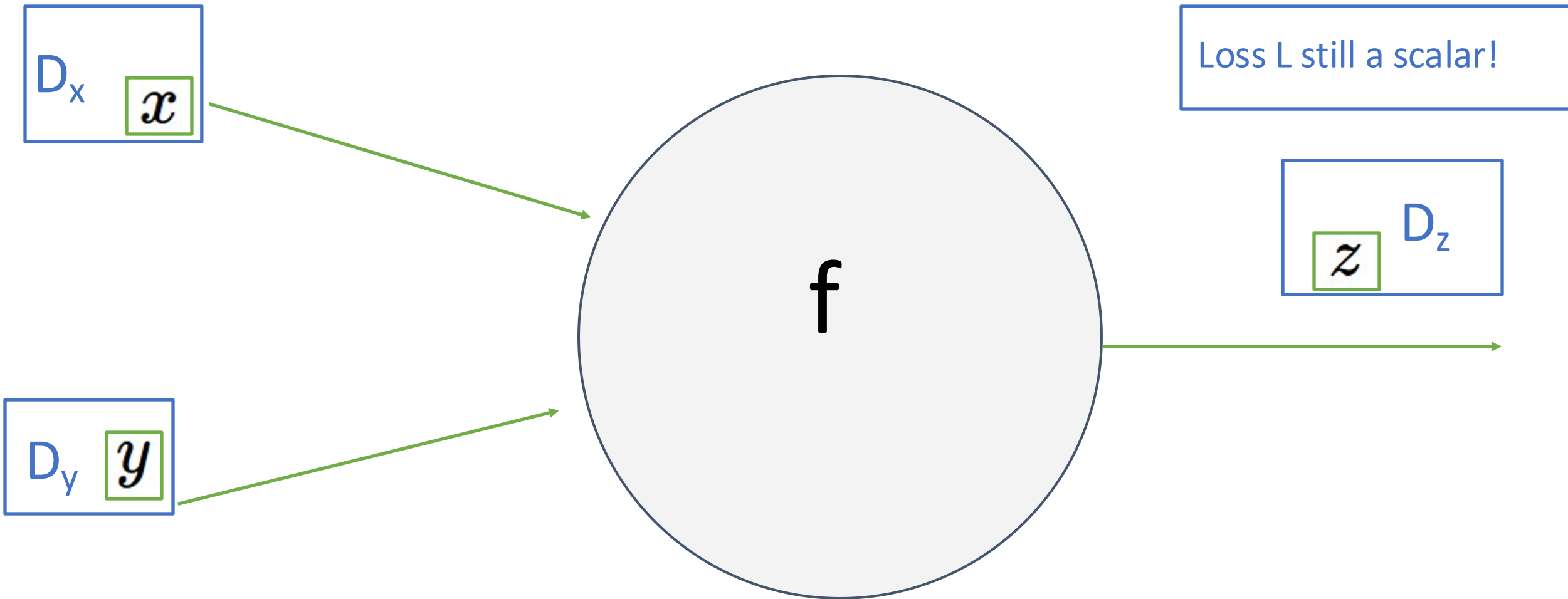
$$x \in \mathbb{R}^N, y \in \mathbb{R}$$

Derivative is **Gradient**:

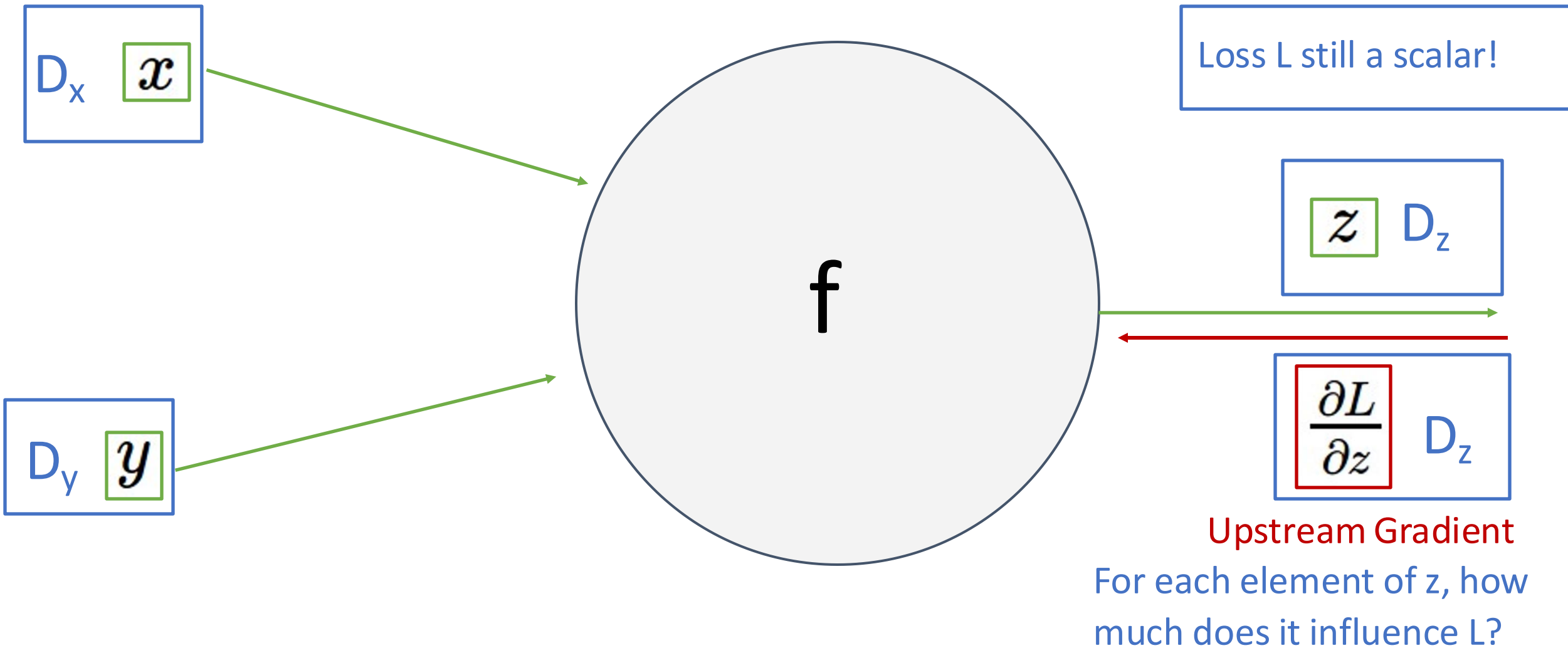
$$\frac{\partial y}{\partial x} \in \mathbb{R}^N \quad \left(\frac{\partial y}{\partial x} \right)_n = \frac{\partial y}{\partial x_n}$$

For each element of x , if it changes by a small amount then how much will y change?

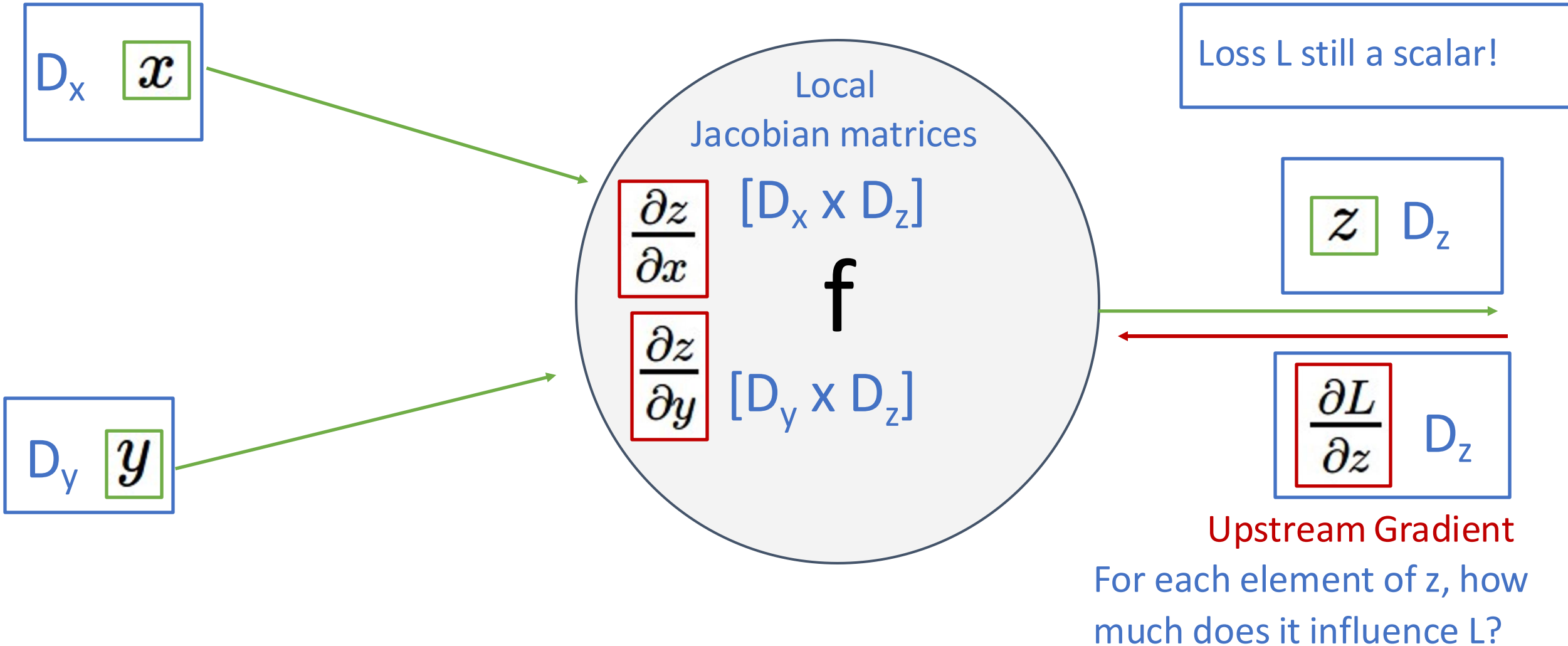
Backprop with Vectors



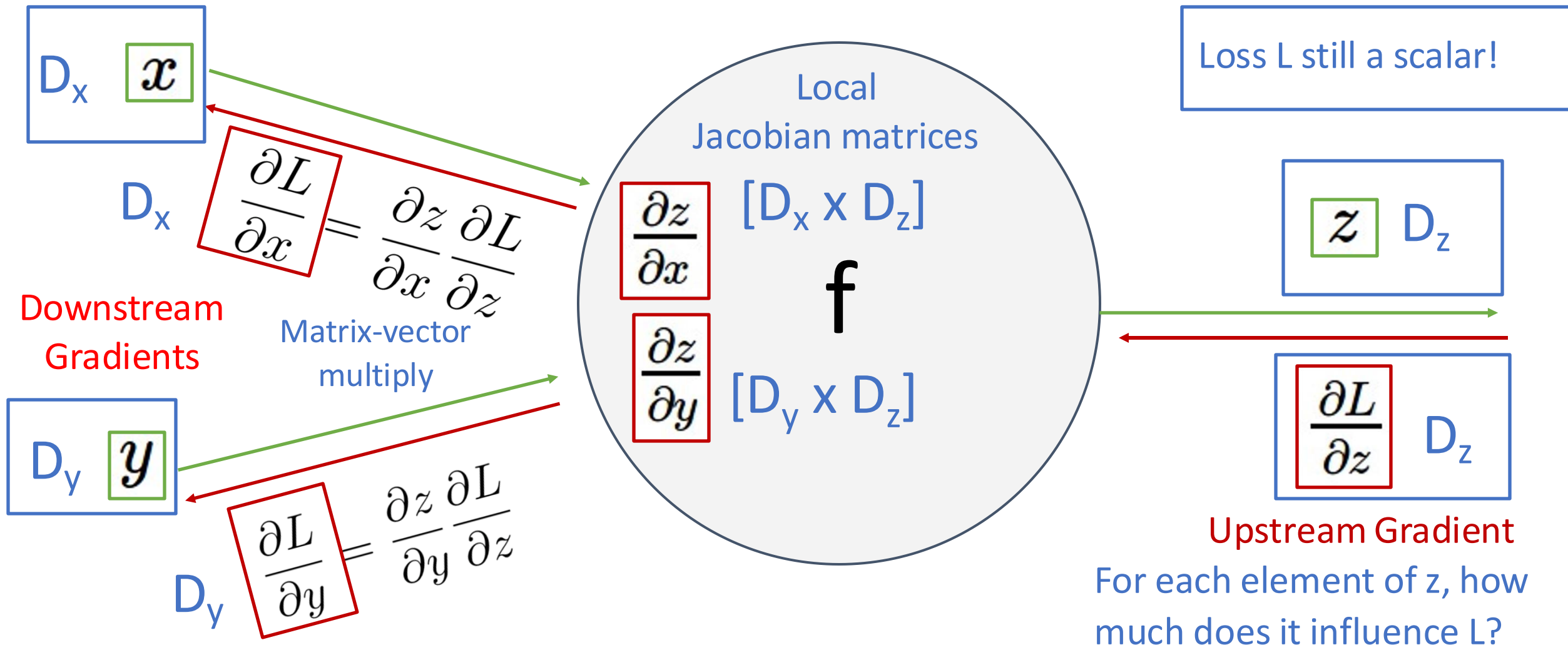
Backprop with Vectors



Backprop with Vectors



Backprop with Vectors



Backprop with Vectors

4D input x:

$\begin{bmatrix} 1 \\ -2 \\ 3 \\ -1 \end{bmatrix}$

$$f(x) = \max(0, x)$$

(elementwise)

4D output y:

$\begin{bmatrix} 1 \\ 0 \\ 3 \\ 0 \end{bmatrix}$

Backprop with Vectors

4D input x:

$\begin{bmatrix} 1 \\ -2 \\ 3 \\ -1 \end{bmatrix}$

$$f(x) = \max(0, x)$$

(elementwise)

4D output y:

$\begin{bmatrix} 1 \\ 0 \\ 3 \\ 0 \end{bmatrix}$

4D dL/dy :

$\begin{bmatrix} 4 \\ -1 \\ 5 \\ 9 \end{bmatrix}$

Upstream
gradient

Backprop with Vectors

4D input x:

$\begin{bmatrix} 1 \\ -2 \\ 3 \\ -1 \end{bmatrix}$

$$f(x) = \max(0, x)$$

(elementwise)

4D output y:

$\begin{bmatrix} 1 \\ 0 \\ 3 \\ 0 \end{bmatrix}$

$\begin{bmatrix} dy/dx & dL/dy \end{bmatrix}$

$\begin{bmatrix} 1 & 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} 4 \end{bmatrix}$

$\begin{bmatrix} 0 & 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} -1 \end{bmatrix}$

$\begin{bmatrix} 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} 5 \end{bmatrix}$

$\begin{bmatrix} 0 & 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} 9 \end{bmatrix}$

4D dL/dy:

$\begin{bmatrix} 4 \end{bmatrix}$

$\begin{bmatrix} -1 \end{bmatrix}$

$\begin{bmatrix} 5 \end{bmatrix}$

$\begin{bmatrix} 9 \end{bmatrix}$

Upstream
gradient

Backprop with Vectors

4D input x:

$\begin{bmatrix} 1 \\ -2 \\ 3 \\ -1 \end{bmatrix}$

$$f(x) = \max(0, x) \\ (\textit{elementwise})$$

4D output y:

$\begin{bmatrix} 1 \\ 0 \\ 3 \\ 0 \end{bmatrix}$

4D dL/dx:

$\begin{bmatrix} 4 \\ 0 \\ 5 \\ 0 \end{bmatrix}$

$\begin{bmatrix} dy/dx & dL/dy \end{bmatrix}$

$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} 4 \\ -1 \\ 5 \\ 9 \end{bmatrix}$

4D dL/dy:

$\begin{bmatrix} 4 \\ -1 \\ 5 \\ 9 \end{bmatrix}$

Upstream
gradient

Very sparse – in practice you don't compute all of it

Backprop with Vectors

Jacobian is **sparse**: off-diagonal entries all zero! Never **explicitly** form Jacobian; instead use **implicit** multiplication

4D input x:

$\begin{bmatrix} 1 \\ -2 \\ 3 \\ -1 \end{bmatrix}$

$f(x) = \max(0, x)$
(*elementwise*)

4D output y:

$\begin{bmatrix} 1 \\ 0 \\ 3 \\ 0 \end{bmatrix}$

4D dL/dx:

$\begin{bmatrix} 4 \\ 0 \\ 5 \\ 0 \end{bmatrix}$

$[dy/dx] [dL/dy]$

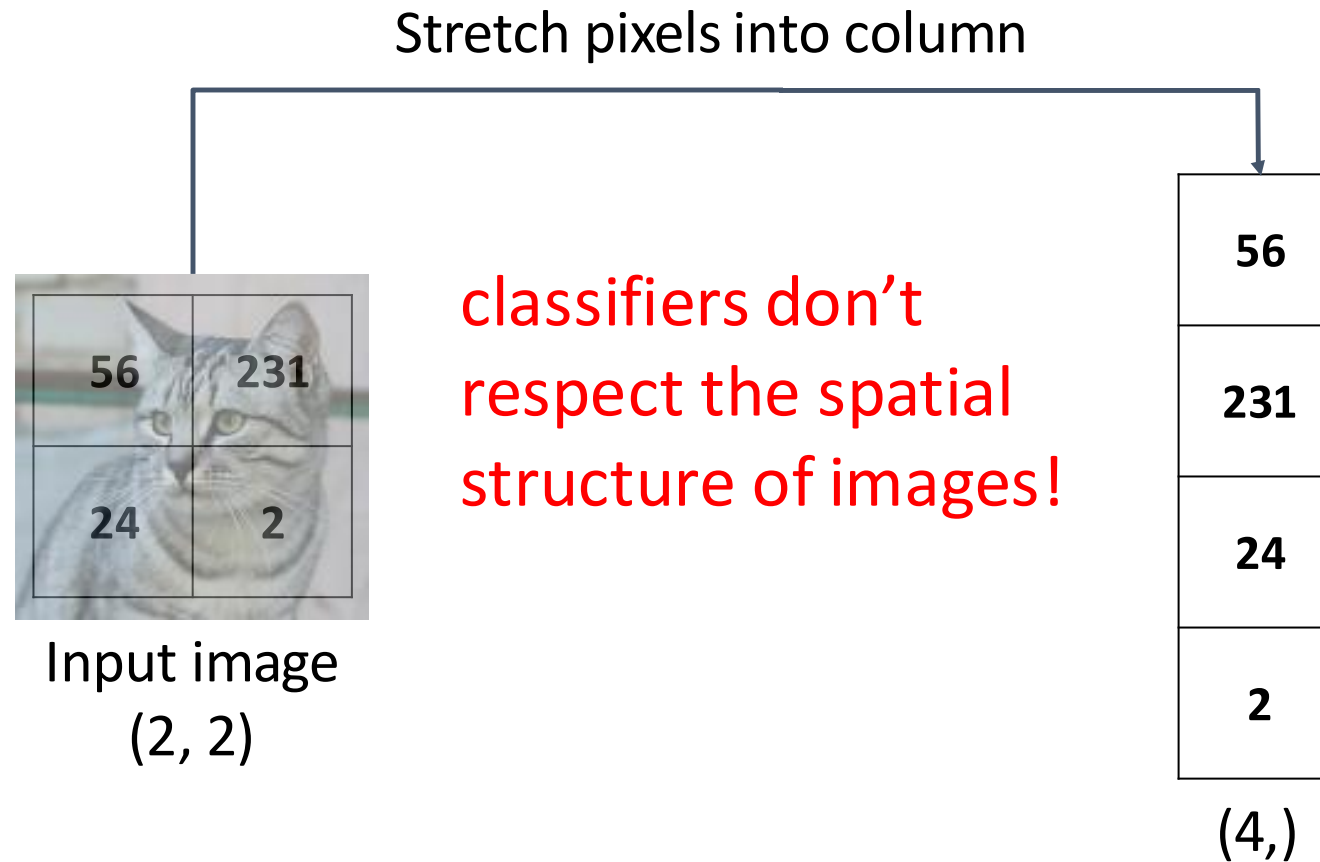
$$\left(\frac{\partial L}{\partial x} \right)_i = \begin{cases} \left(\frac{\partial L}{\partial y} \right)_i & \text{if } x_i > 0 \\ 0 & \text{otherwise} \end{cases}$$

4D dL/dy:

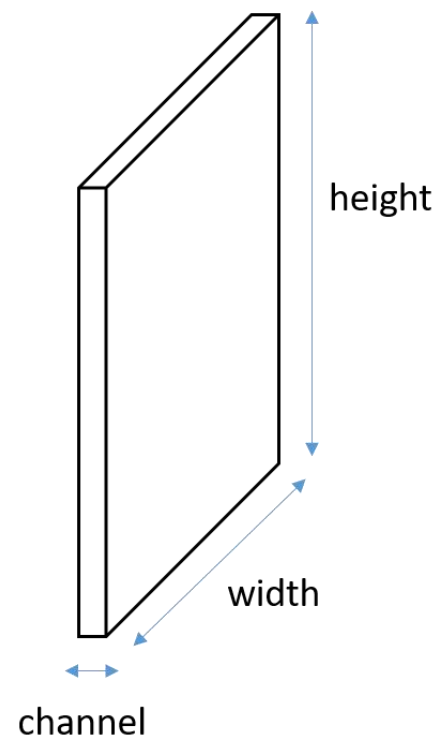
$\begin{bmatrix} 4 \\ -1 \\ 5 \\ 9 \end{bmatrix}$

Upstream
gradient

Are MLP good for images?

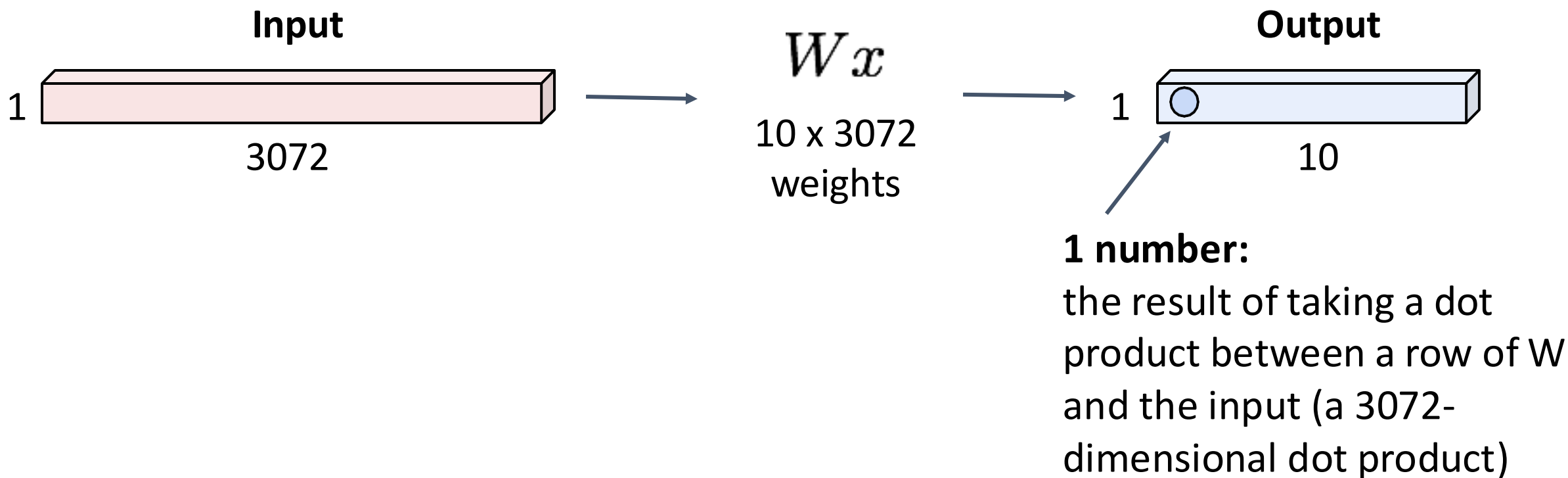


What is an image?



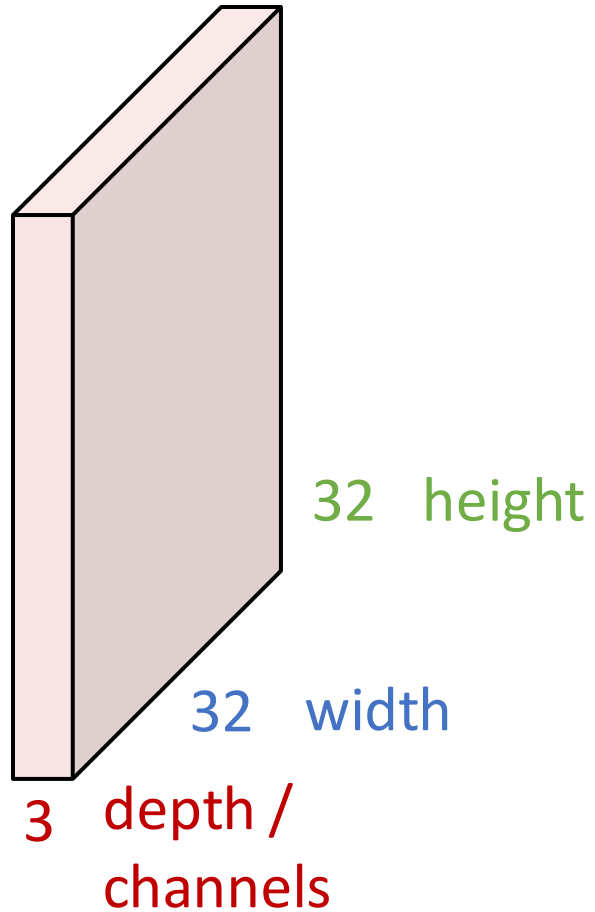
Fully-Connected Layer

32x32x3 image -> stretch to 3072 x 1



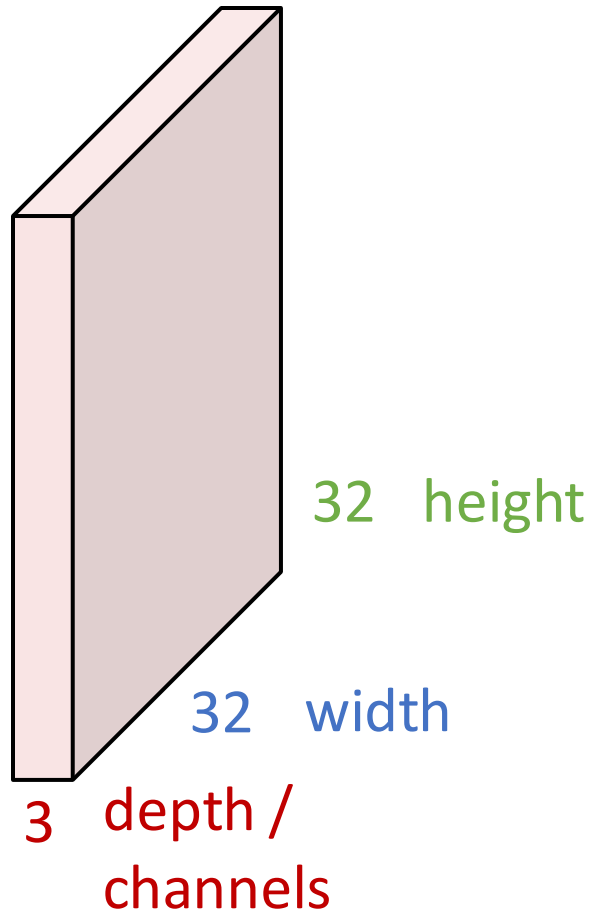
Convolution Layer

3x32x32 image: preserve spatial structure

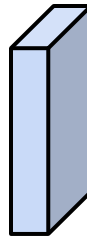


Convolution Layer

3x32x32 image: preserve spatial structure



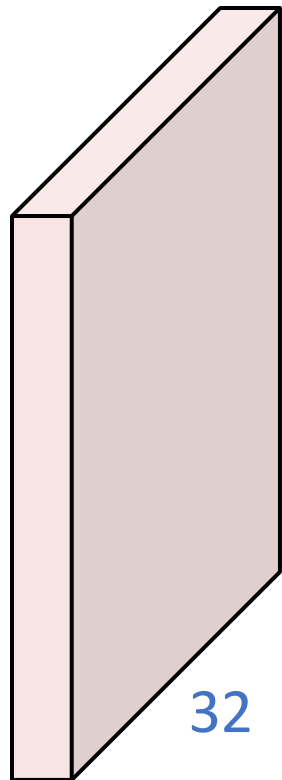
3x5x5 filter



Convolve the filter with the image
i.e. “slide over the image spatially,
computing dot products”

Convolution Layer

3x32x32 image



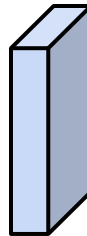
32 height

32 width

3 depth /
channels

Filters always extend the full
depth of the input volume

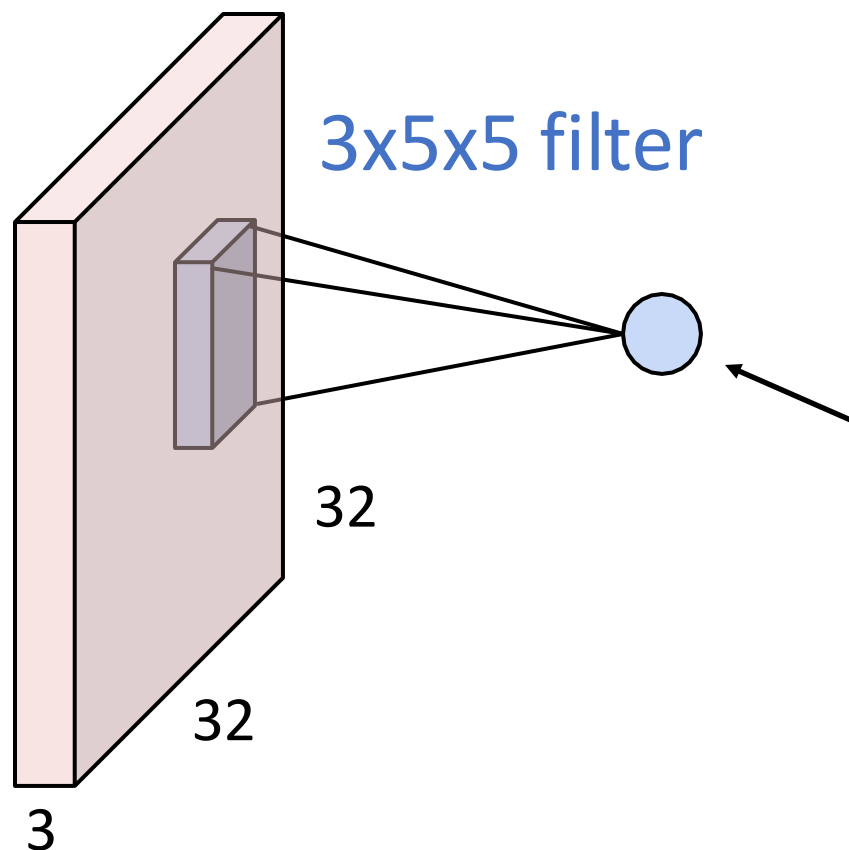
3x5x5 filter



Convolve the filter with the image
i.e. “slide over the image spatially,
computing dot products”

Convolution Layer

3x32x32 image

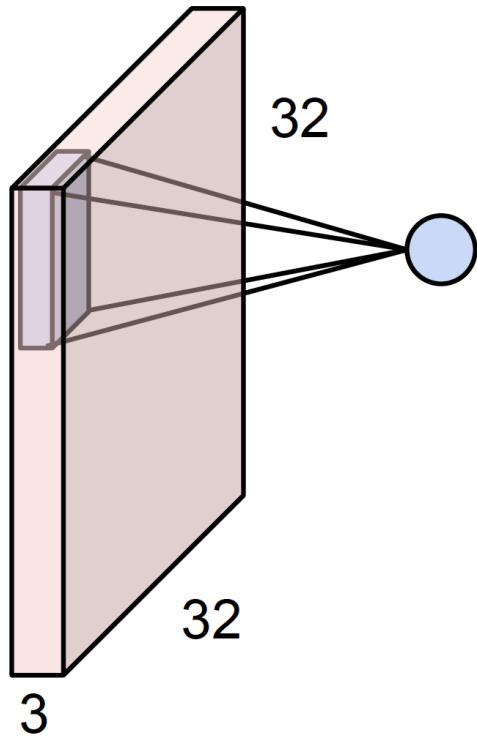


1 number:

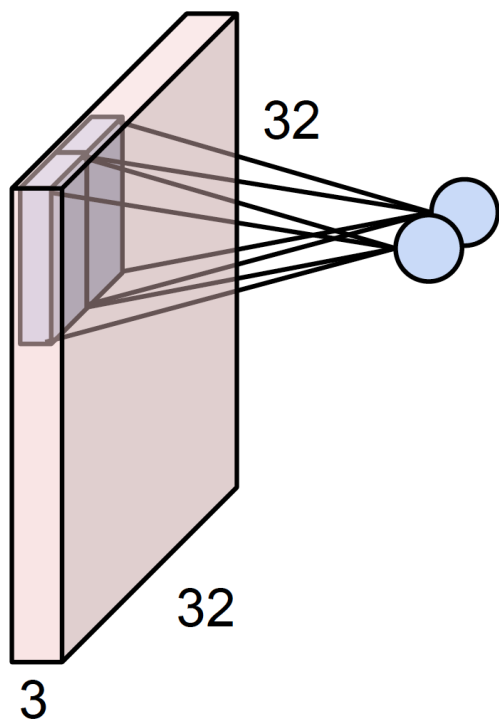
the result of taking a dot product between the filter
and a small 3x5x5 chunk of the image
(i.e. $3 \times 5 \times 5 = 75$ -dimensional dot product + bias)

$$w^T x + b$$

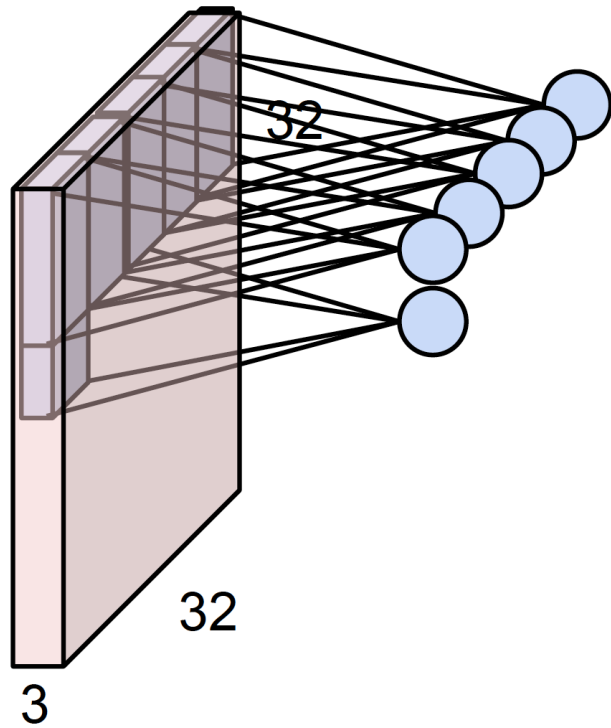
Convolutional Layer



Convolutional Layer



Convolutional Layer



Running example

- For example, let's take the filter

1	0	1
0	1	0
1	0	1

1x1	1x0	1x1	0	0
0x0	1x1	1x0	1	0
0x1	0x0	1x1	1	1
0	0	1	1	0
0	1	1	0	0

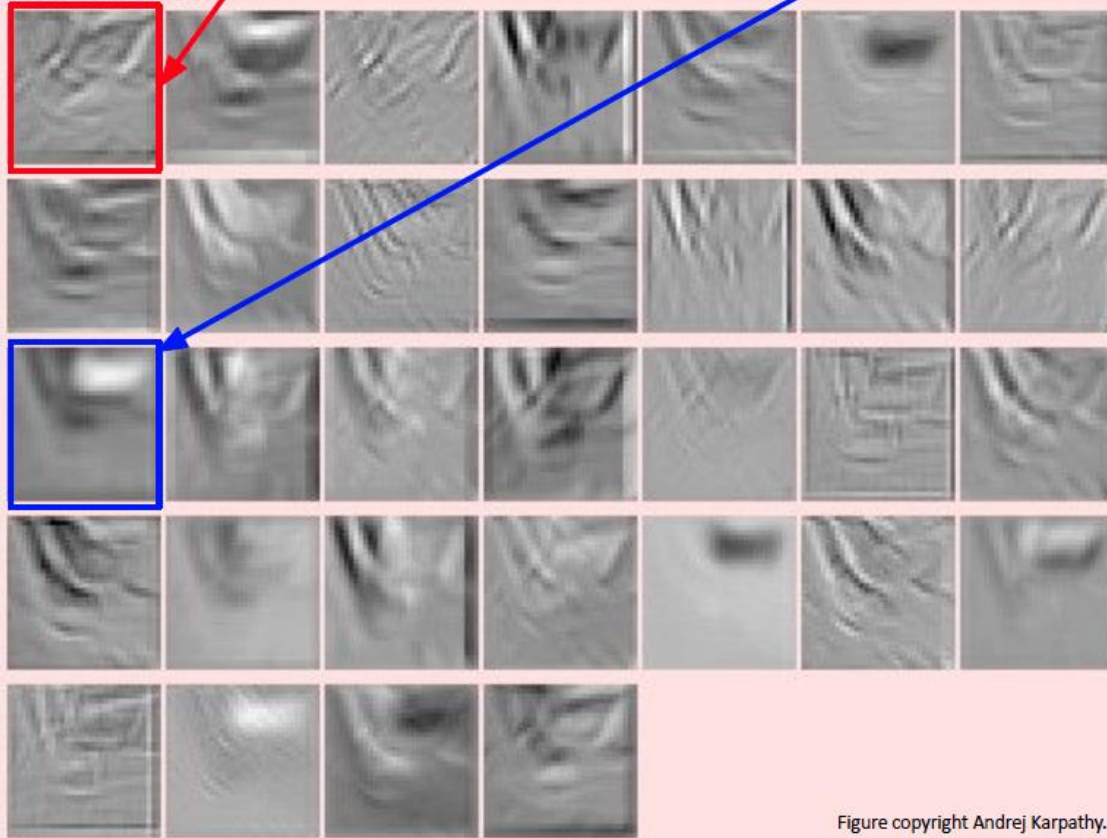
4		



one filter =>
one activation map



Activations:



example 5x5 filters
(32 total)

Known filters (Kernels)



$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$



Identity

$$* \begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix} =$$



Edge detection

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

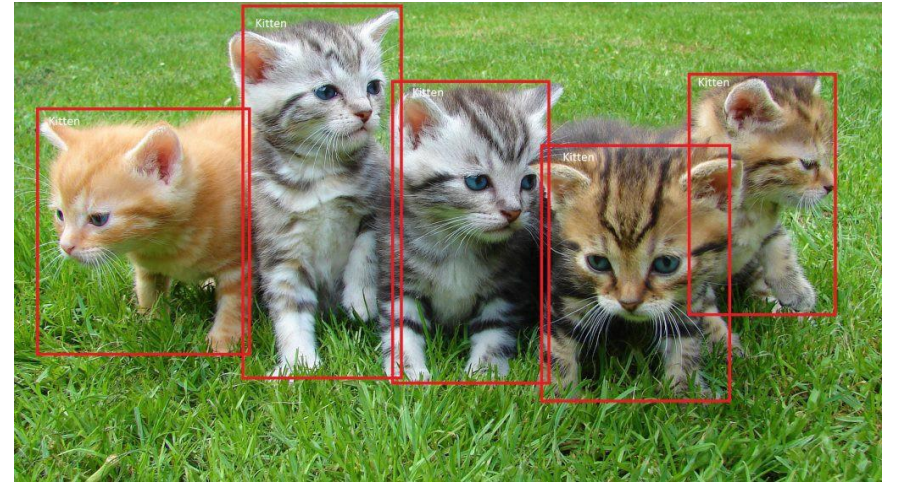


Sharpen

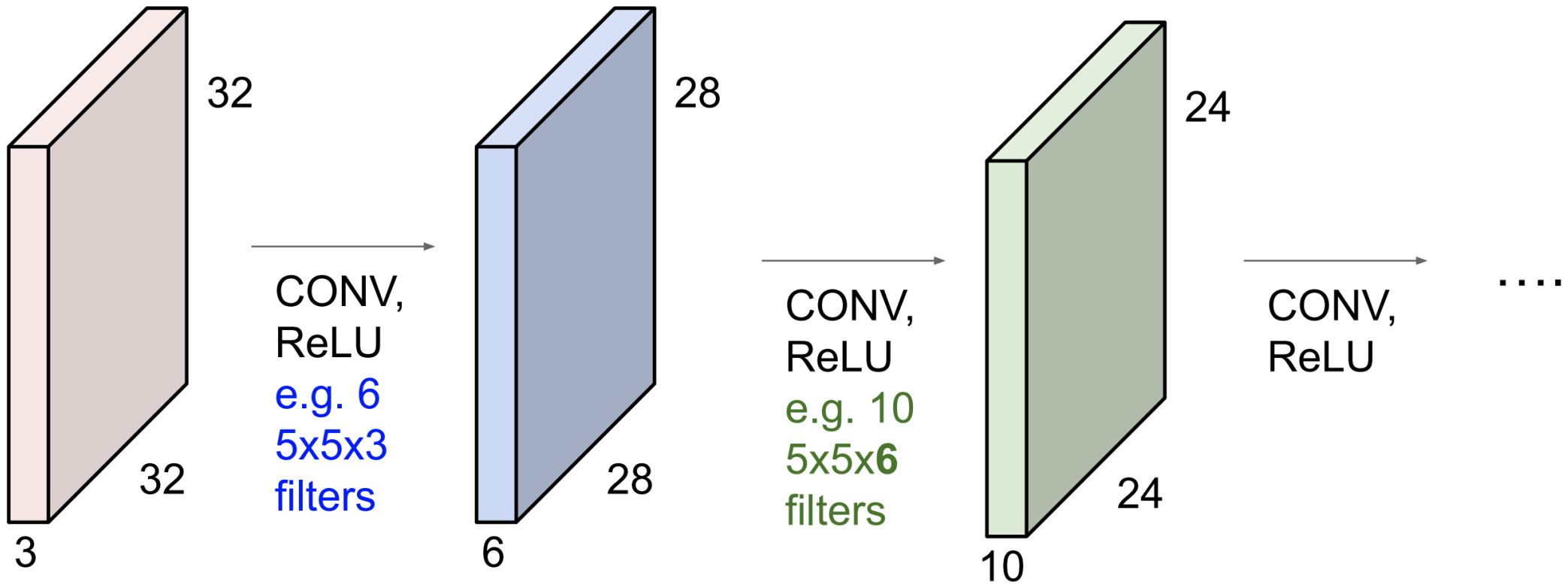
- Intuitively Neural Networks – **We learn the best kernels for our task**

Translational equivariance

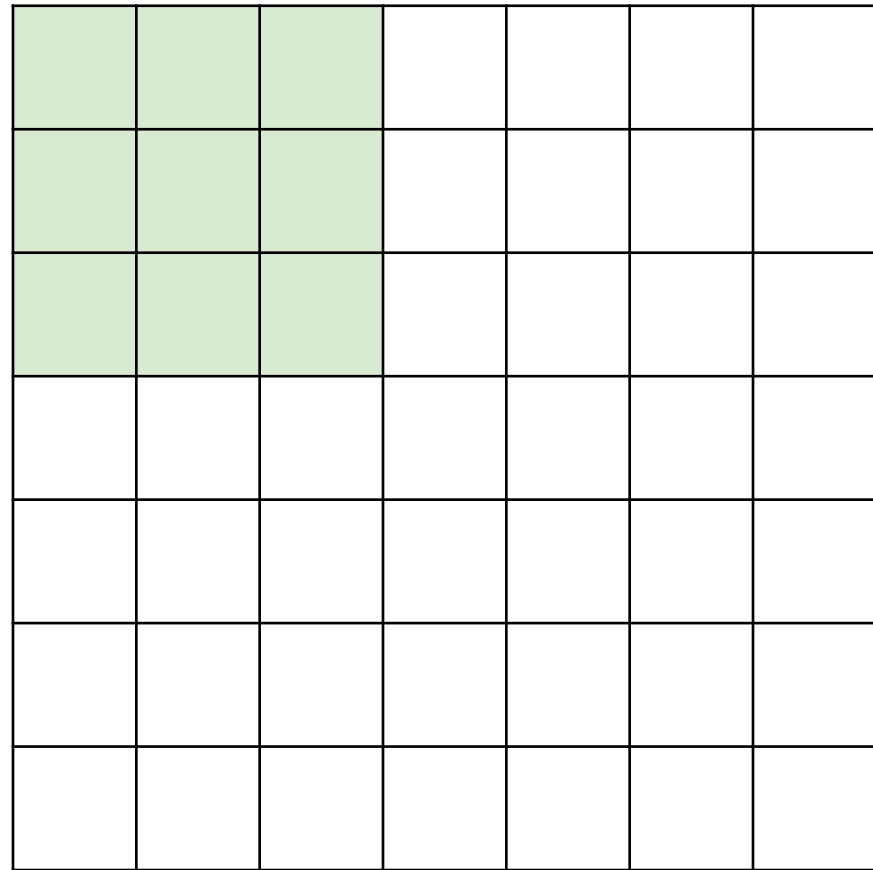
- The filter detecteds patterns irrespective of position in the image.



ConvNet



A closer look at spatial dimensions



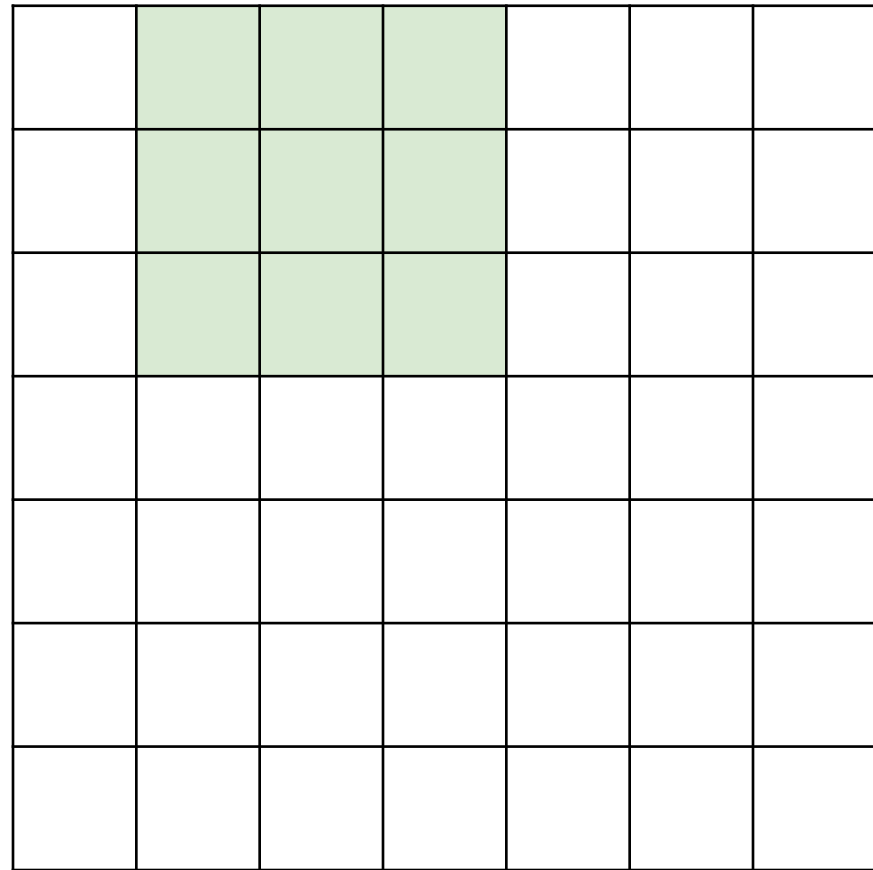
Input: 7x7

Filter: 3x3

7

7

A closer look at spatial dimensions



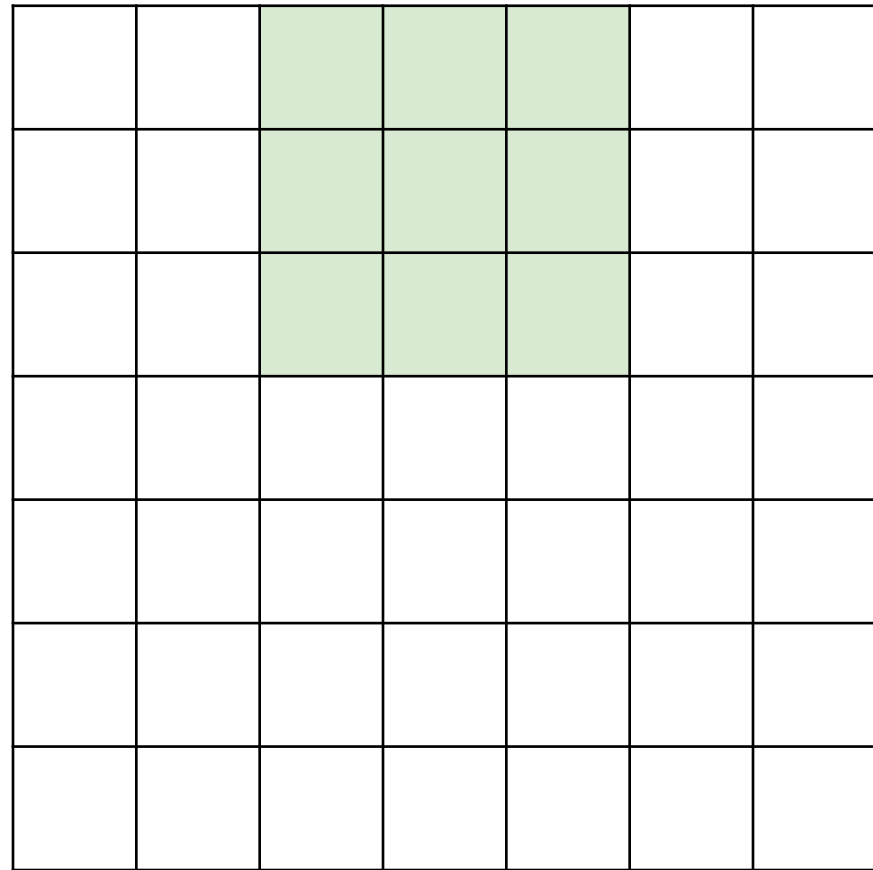
7

7

Input: 7x7

Filter: 3x3

A closer look at spatial dimensions



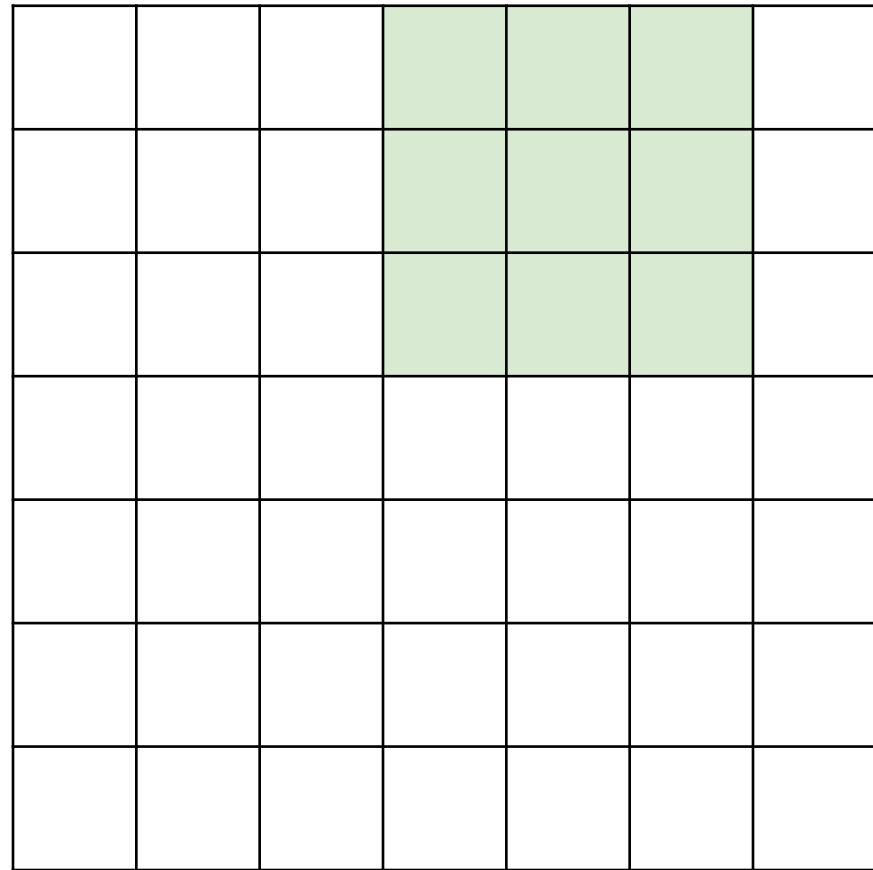
7

7

Input: 7x7

Filter: 3x3

A closer look at spatial dimensions



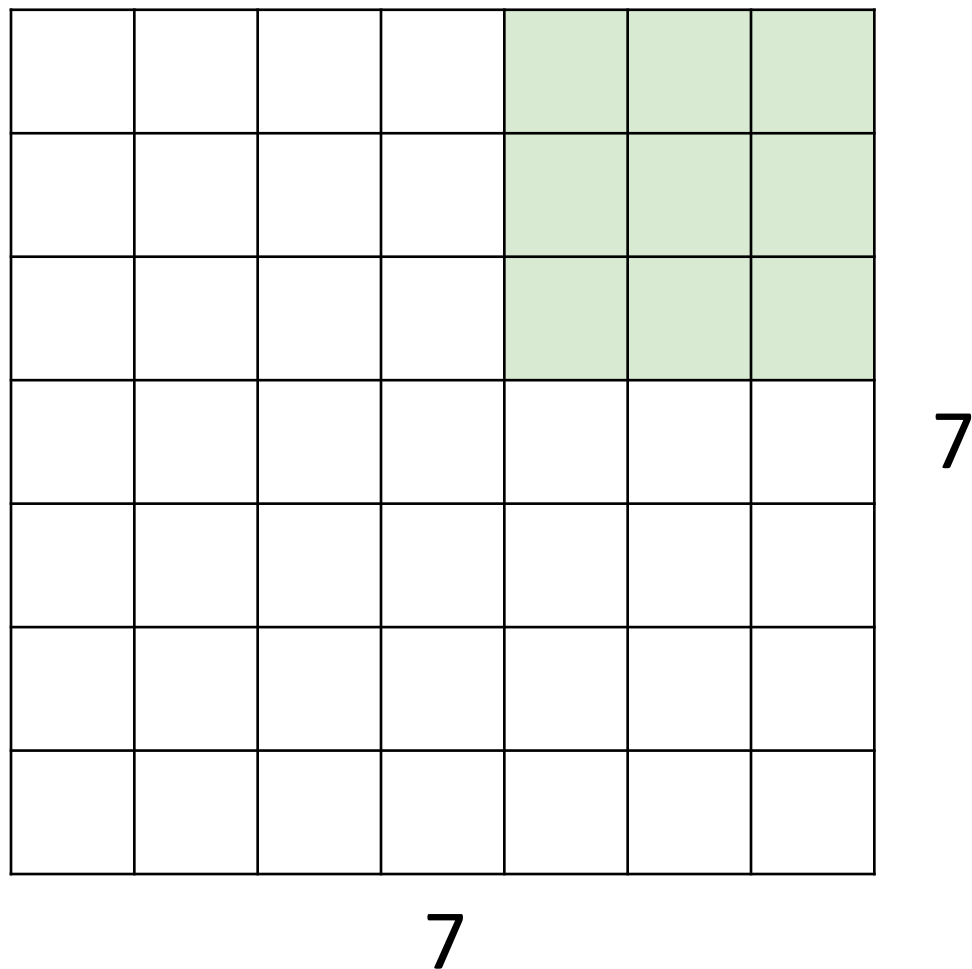
7

7

Input: 7x7

Filter: 3x3

A closer look at spatial dimensions

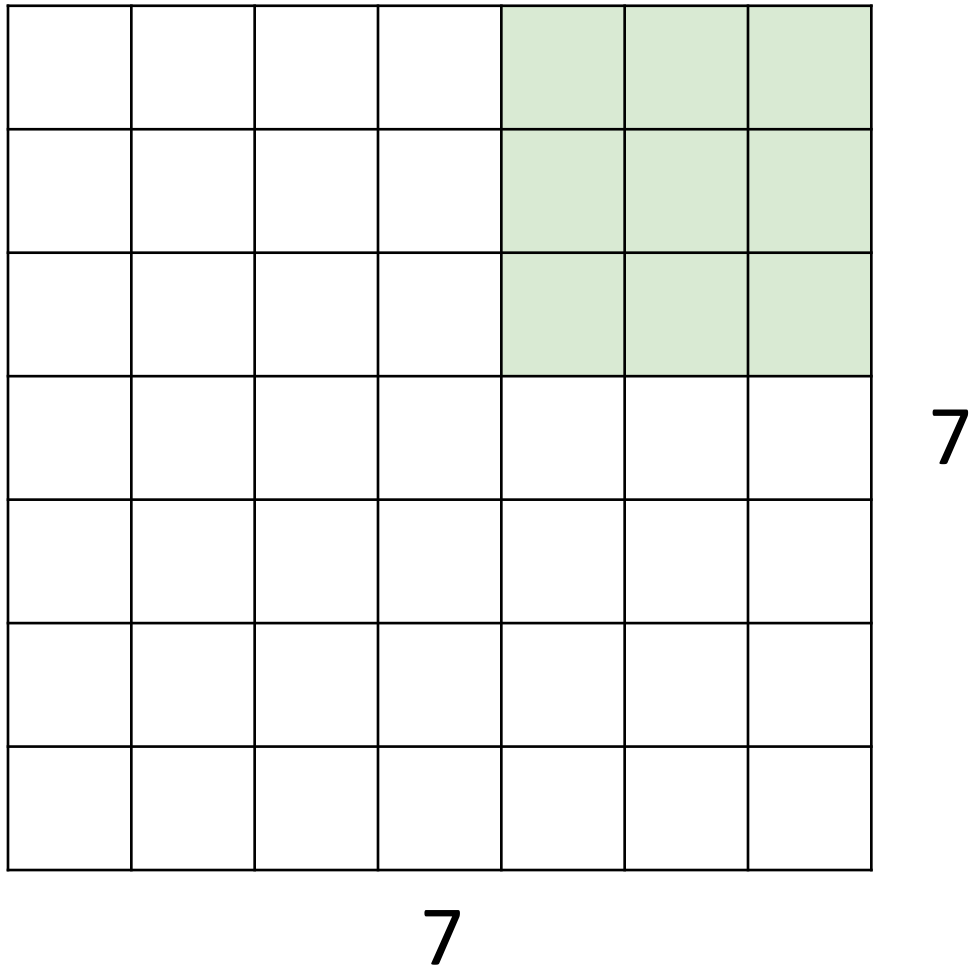


Input: 7x7

Filter: 3x3

Output: 5x5

A closer look at spatial dimensions



Input: 7x7

Filter: 3x3

Output: 5x5

Problem: Feature
maps “shrink”
with each layer!

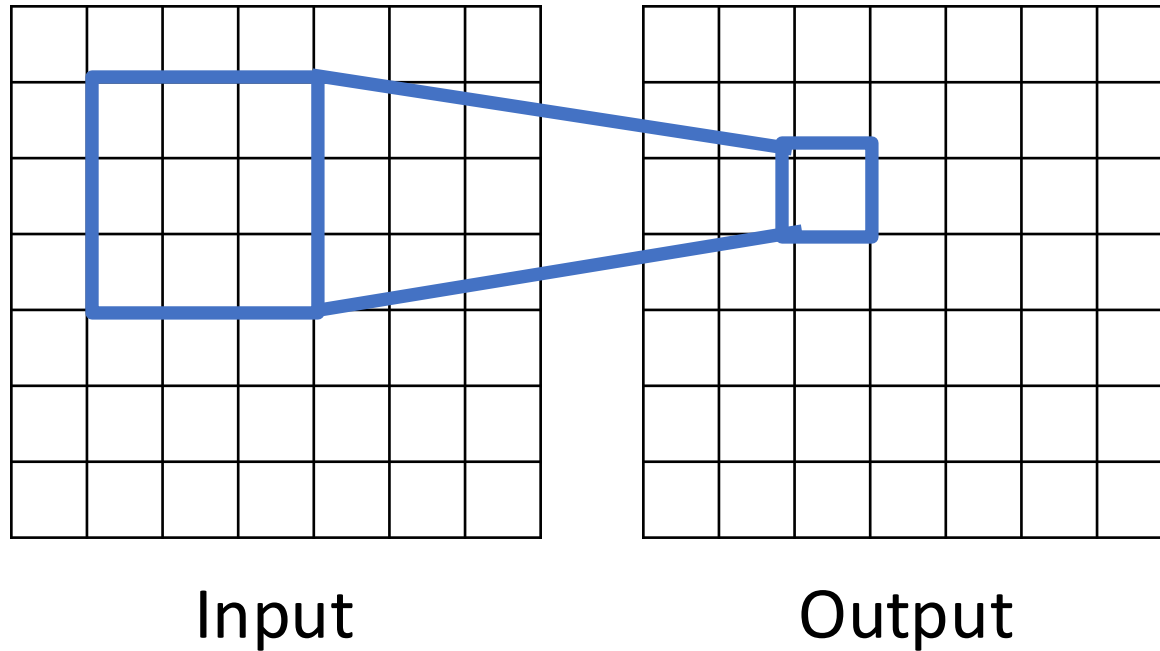
Commonly: add padding

0	0	0	0	0	0	0	0	0
0								0
0								0
0								0
0								0
0								0
0								0
0								0
0	0	0	0	0	0	0	0	0

Very common:
Set $P = (K - 1) / 2$ to
make output have
same size as input!

Receptive Fields

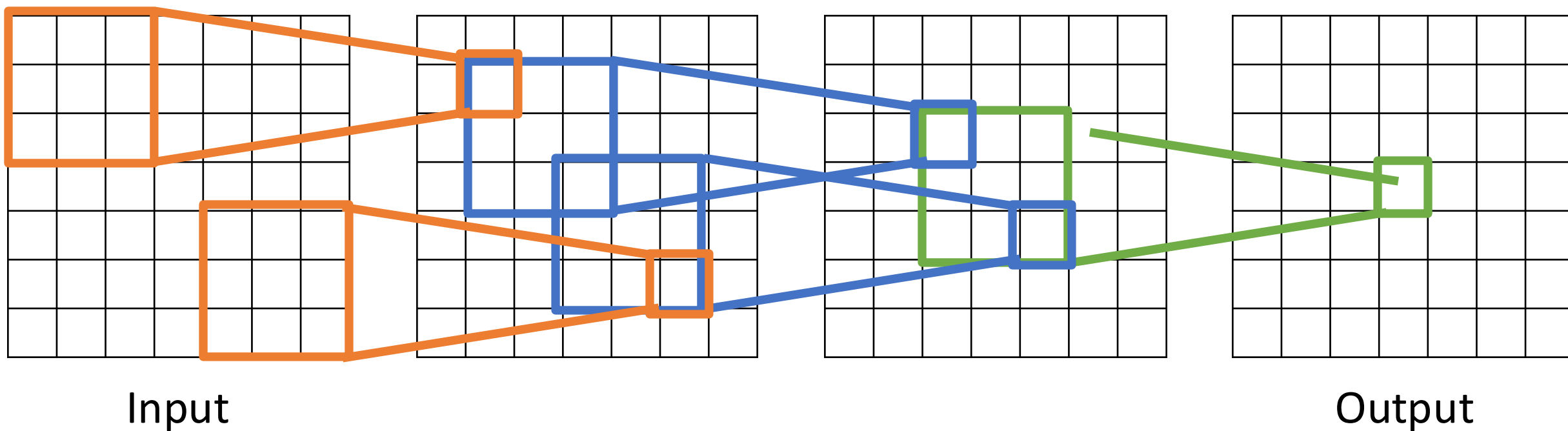
For convolution with kernel size K , each element in the output depends on a $K \times K$ **receptive field** in the input



Receptive Fields

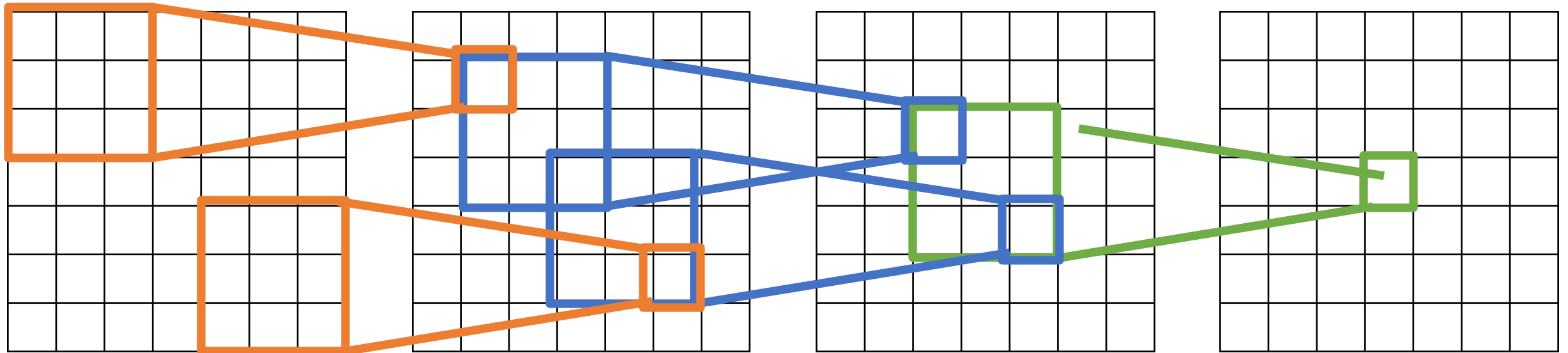
Each successive convolution adds $K - 1$ to the receptive field size

With L layers the receptive field size is $1 + L * (K - 1)$



Receptive Fields

Each successive convolution adds $K - 1$ to the receptive field size
With L layers the receptive field size is $1 + L * (K - 1)$



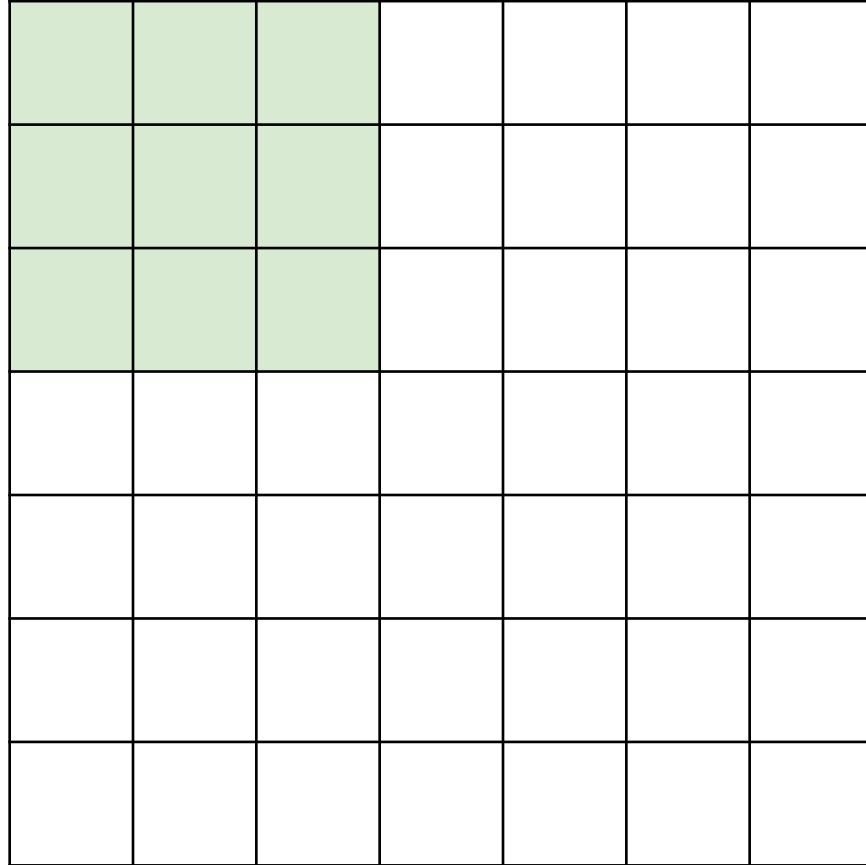
Input

Problem: For large images we need many layers
for each output to “see” the whole image

Solution: Downsample inside the network

Output

Strided Convolution

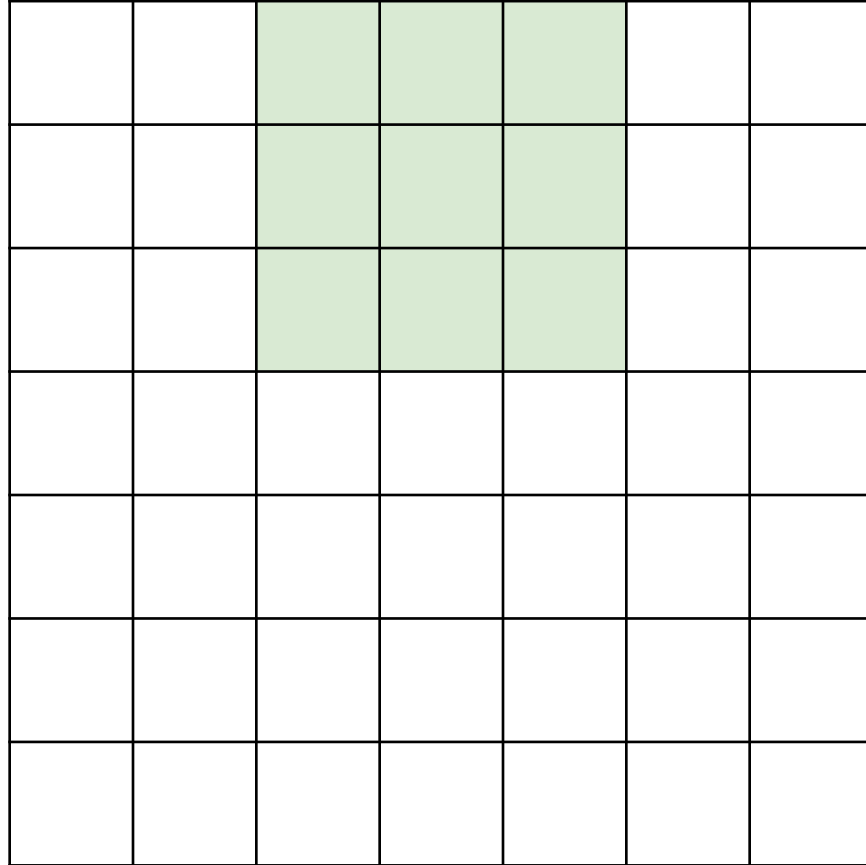


Input: 7x7

Filter: 3x3

Stride: 2

Strided Convolution



Input: 7x7

Filter: 3x3

Stride: 2

Strided Convolution

Input: 7x7

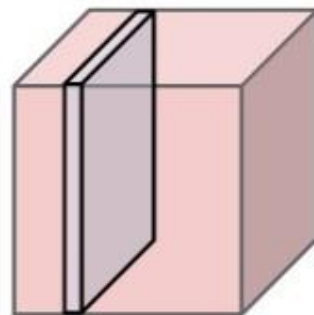
Filter: 3x3

Stride: 2

Output: 3x3

Max Pooling

224x224x64



Single depth slice

x ↑

1	1	2	4
5	6	7	8
3	2	1	0
1	2	3	4

→ y

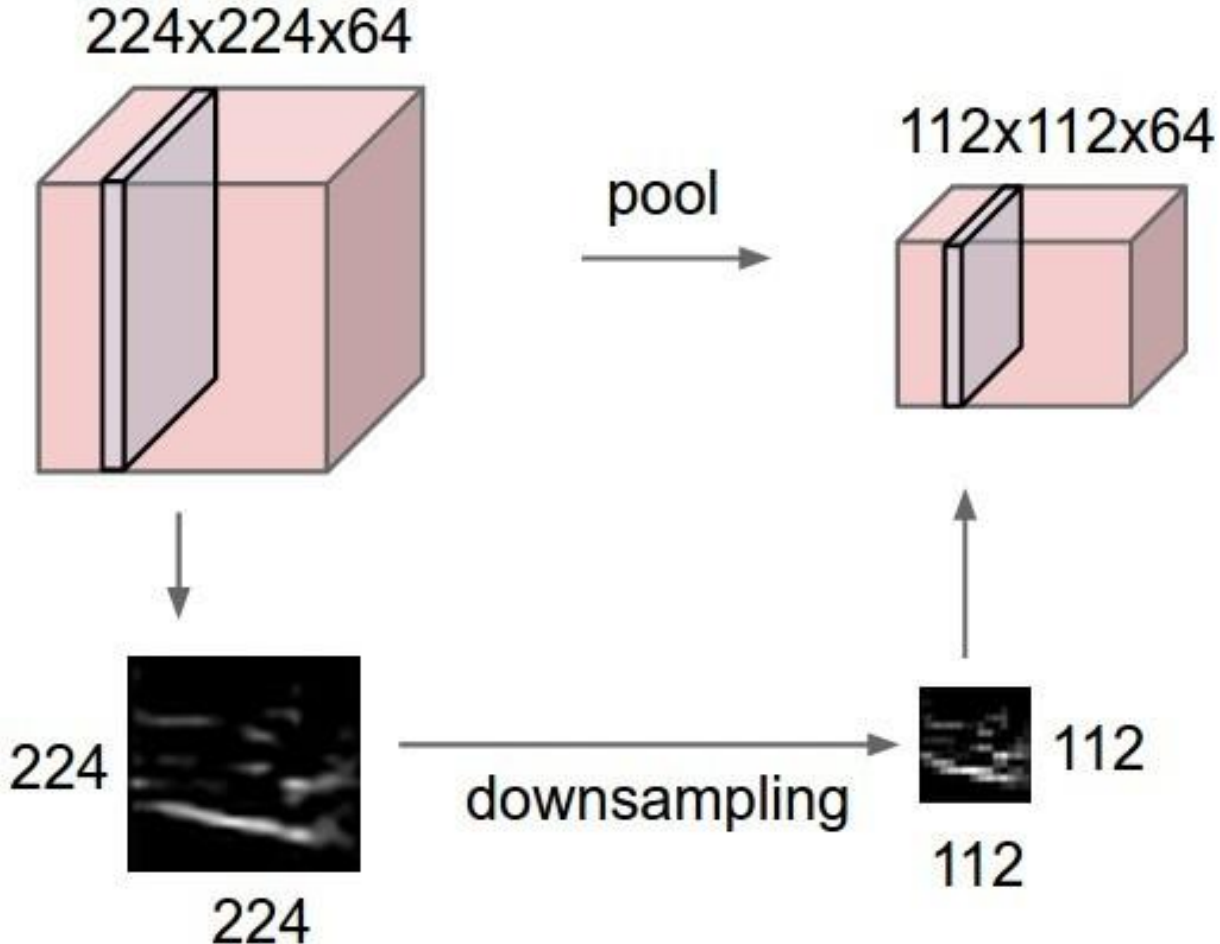
Max pooling with 2x2
kernel size and stride 2



6	8
3	4

Introduces **invariance** to
small spatial shifts

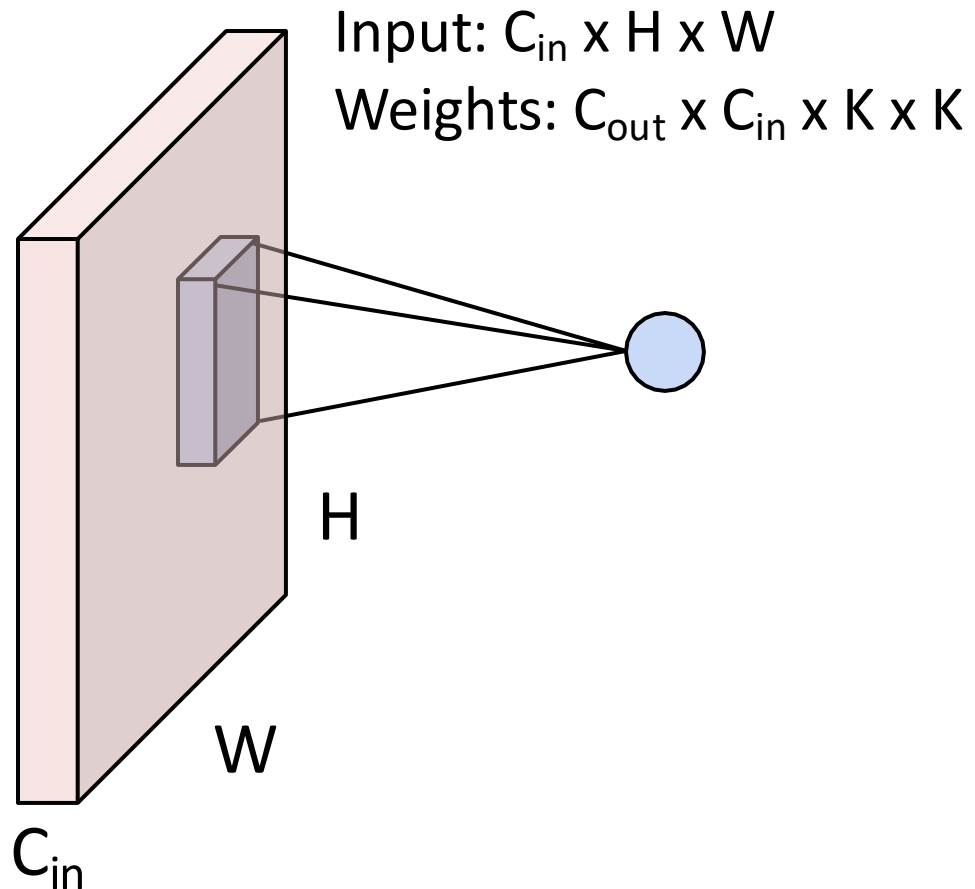
Pooling Layers: Another way to downsample



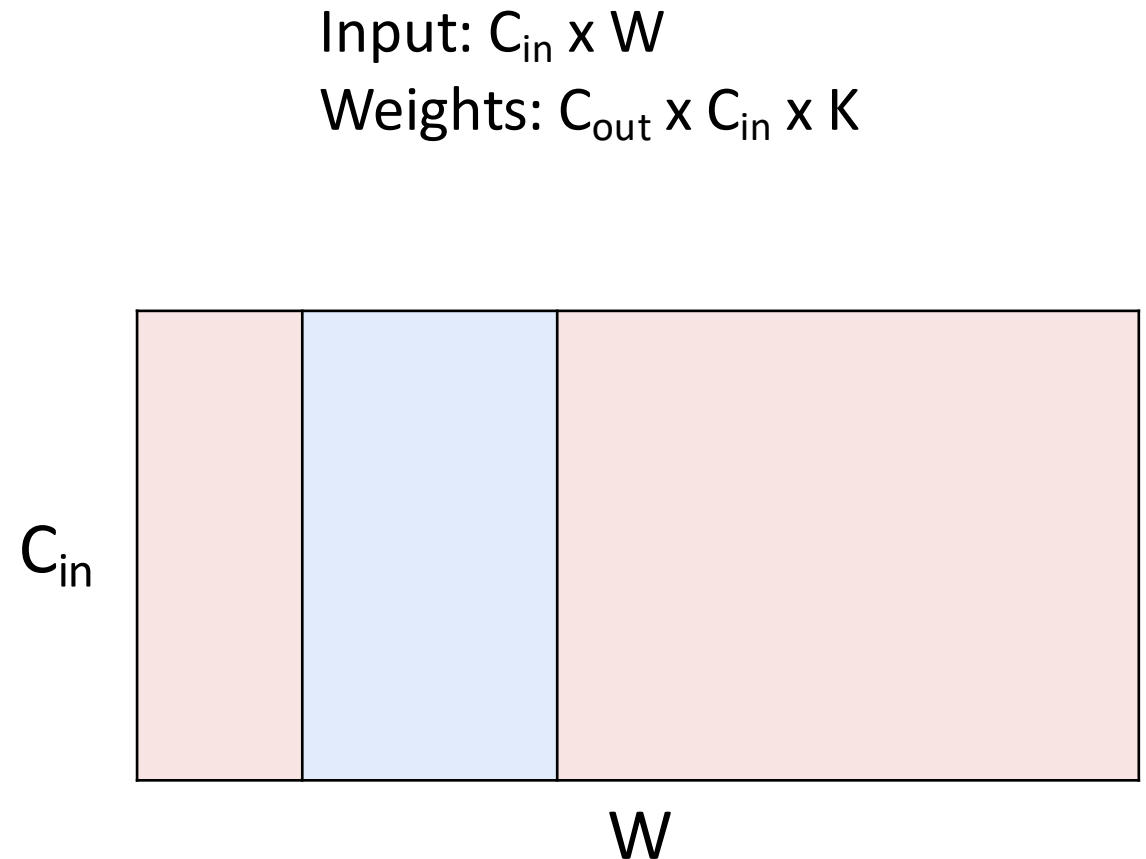
Hyperparameters:
Kernel Size
Stride
Pooling function

1D filters for sequences

So far: 2D Convolution



1D Convolution

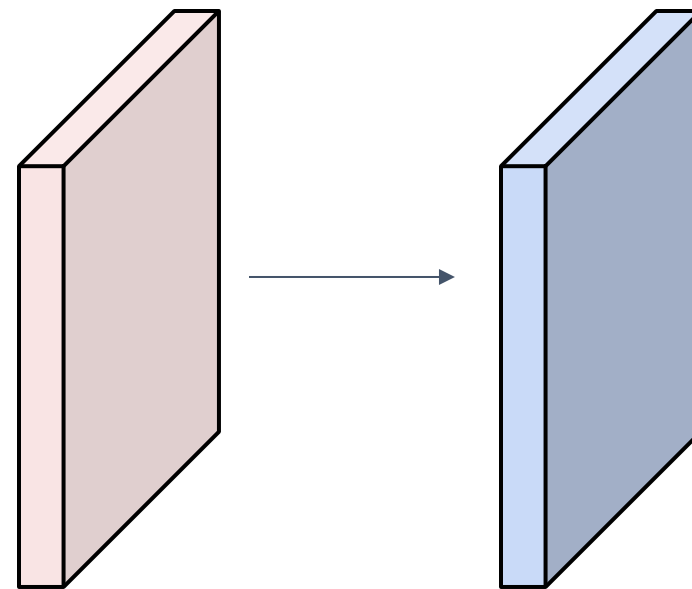


Convolution Example

Input volume: 3 x 32 x 32

10 5x5 filters with stride 1, pad 2

Output volume size: ?



Convolution Example

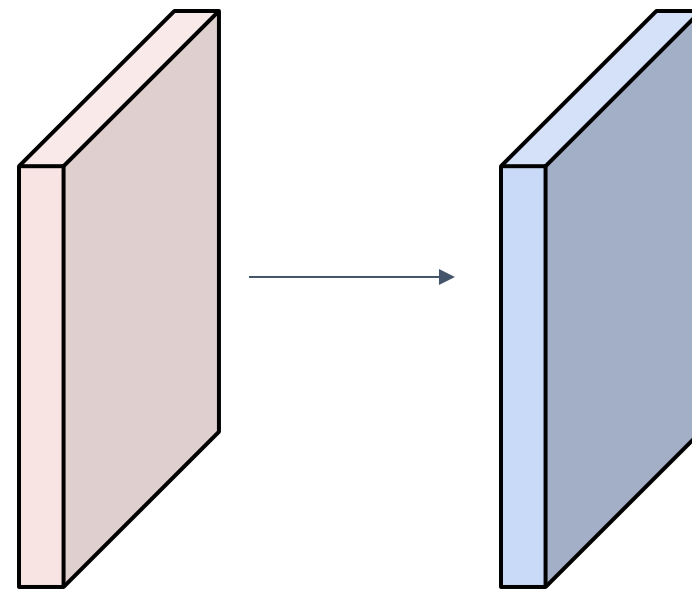
Input volume: 3 x 32 x 32

10 5x5 filters with stride 1, pad 2

Output volume size:

$(32 + 2 * 2 - 5) / 1 + 1 = 32$ spatially, so

10 x 32 x 32



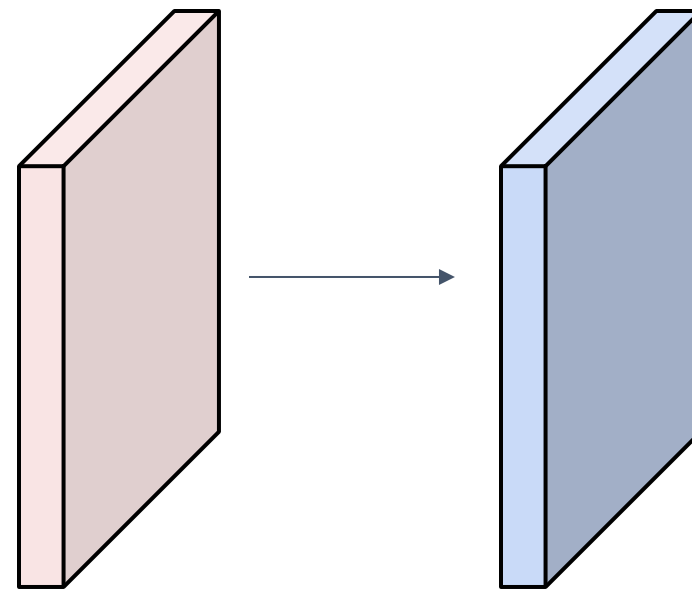
Convolution Example

Input volume: $3 \times 32 \times 32$

10 5×5 filters with stride 1, pad 2

Output volume size: $10 \times 32 \times 32$

Number of learnable parameters: ?



Convolution Example

Input volume: **3** x 32 x 32

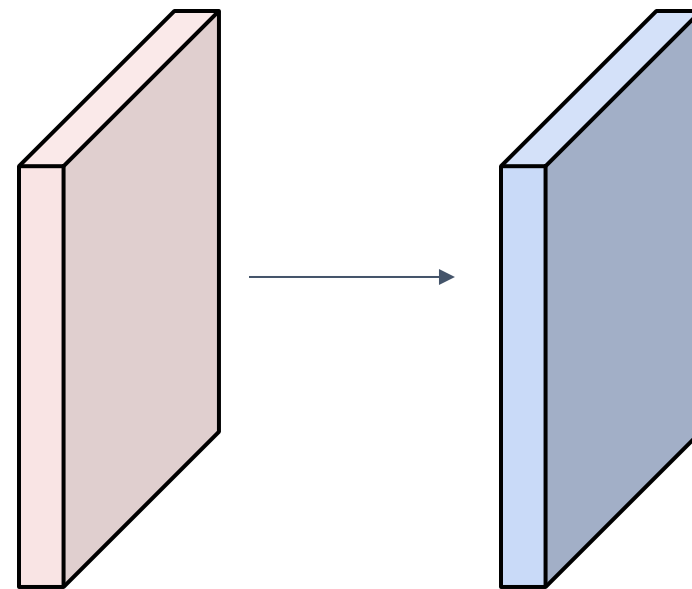
10 **5x5** filters with stride 1, pad 2

Output volume size: 10 x 32 x 32

Number of learnable parameters: **760**

Parameters per filter: **3*****5*****5** + 1 (for bias) = **76**

10 filters, so total is **10** * **76** = **760**



Convolution Example

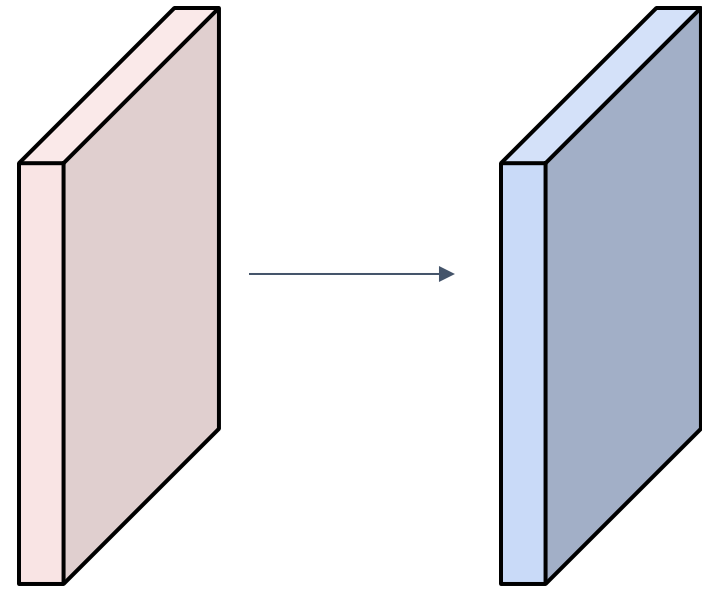
Input volume: $3 \times 32 \times 32$

10 5×5 filters with stride 1, pad 2

Output volume size: $10 \times 32 \times 32$

Number of learnable parameters: 760

Number of multiply-add operations: ?



Convolution Example

Input volume: **3** x 32 x 32

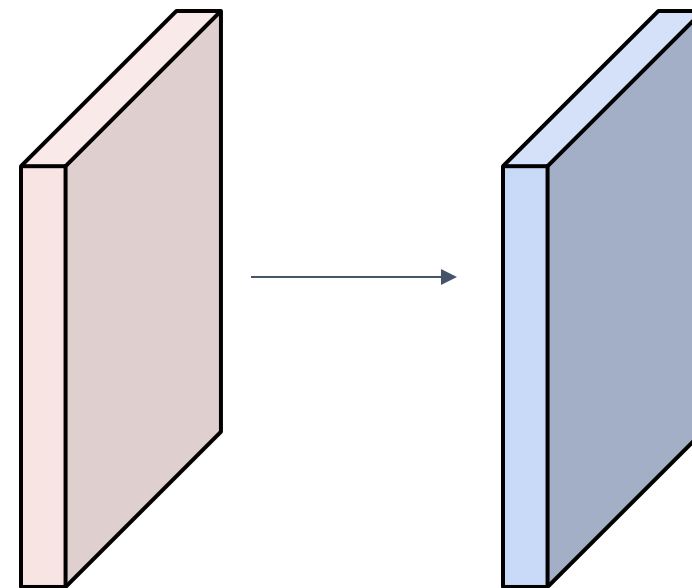
10 **5x5** filters with stride 1, pad 2

Output volume size: **10 x 32 x 32**

Number of learnable parameters: 760

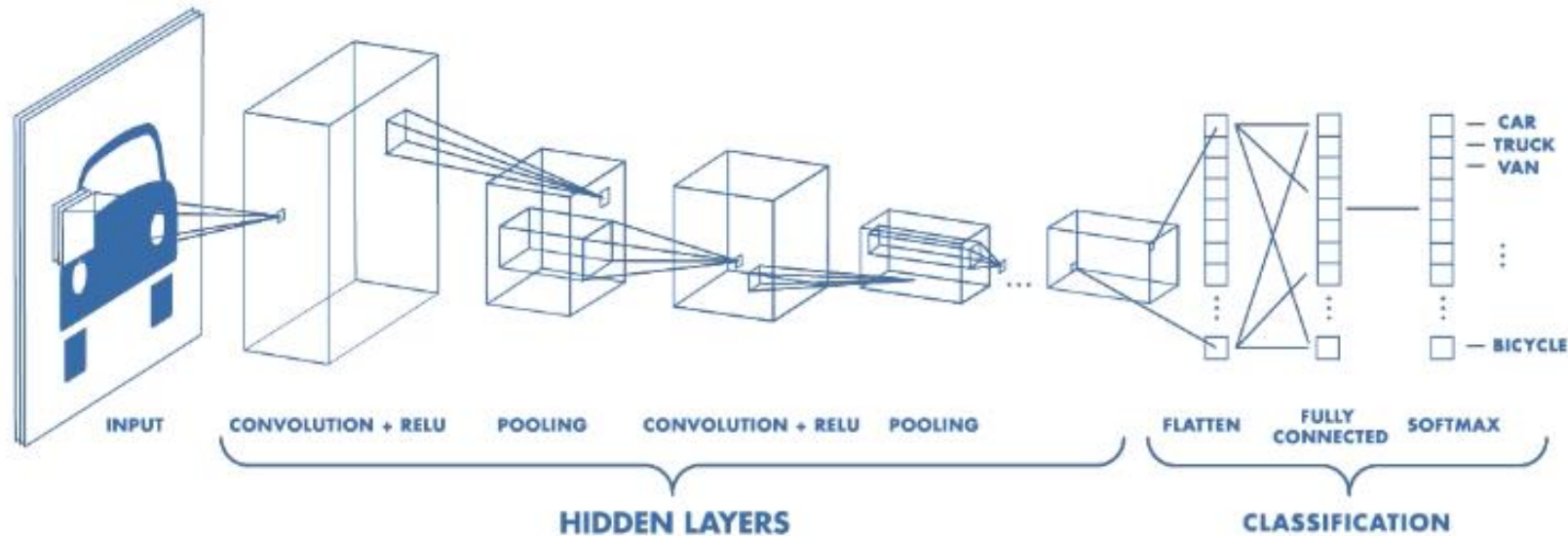
Number of multiply-add operations: **768,000**

10*32*32 = 10,240 outputs; each output is the inner product of two **3x5x5** tensors (75 elems); total = $75 * 10240 = \mathbf{768K}$



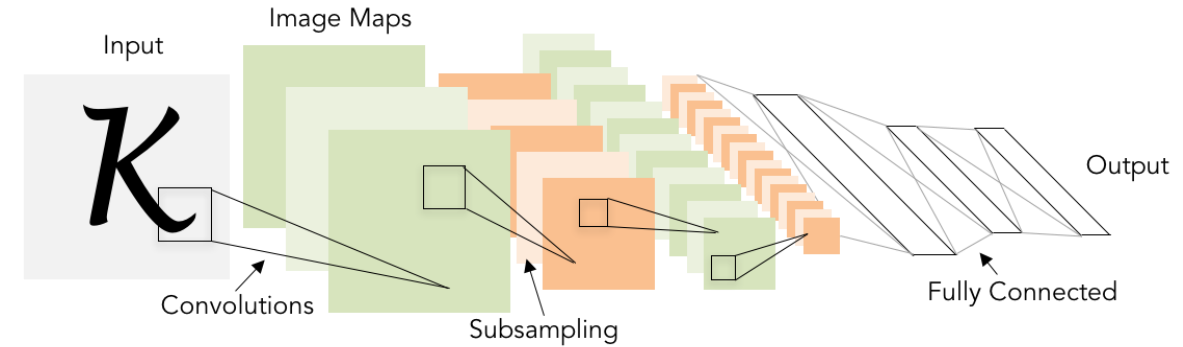
Deep Convolutional Neural Network

- We “scan” the image (convolutions) and aggregate areas (pooling)
- We get a low-dimension representation of the image
- Finally, we use a fully connected to make decisions



Example: LeNet-5

Layer	Output Size	Weight Size
Input	1 x 28 x 28	
Conv ($C_{out}=20$, $K=5$, $P=2$, $S=1$)	20 x 28 x 28	20 x 1 x 5 x 5
ReLU	20 x 28 x 28	
MaxPool($K=2$, $S=2$)	20 x 14 x 14	
Conv ($C_{out}=50$, $K=5$, $P=2$, $S=1$)	50 x 14 x 14	50 x 20 x 5 x 5
ReLU	50 x 14 x 14	
MaxPool($K=2$, $S=2$)	50 x 7 x 7	
Flatten	2450	
Linear (2450 -> 500)	500	2450 x 500
ReLU	500	
Linear (500 -> 10)	10	500 x 10



As we go through the network:

Spatial size **decreases**
(using pooling or strided conv)

Number of channels **increases**
(total “volume” is preserved!)

Batch Normalization

Idea: “Normalize” the outputs of a layer so they have zero mean and unit variance

Why? Helps reduce “internal covariate shift”, improves optimization

We can normalize a batch of activations like this:

$$\hat{x}^{(k)} = \frac{x^{(k)} - \mathbb{E}[x^{(k)}]}{\sqrt{\text{Var}[x^{(k)}]}}$$

This is a **differentiable function**, so we can use it as an operator in our networks and backprop through it!

Batch Normalization

Input: $x : N \times D$

Learnable scale and shift parameters:

$$\gamma, \beta : D$$

Learning $\gamma = \sigma$,
 $\beta = \mu$ will recover the
identity function!

$$\mu_j = \frac{1}{N} \sum_{i=1}^N x_{i,j}$$

Per-channel
mean, shape is D

$$\sigma_j^2 = \frac{1}{N} \sum_{i=1}^N (x_{i,j} - \mu_j)^2$$

Per-channel
std, shape is D

$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \varepsilon}}$$

Normalized x,
Shape is N x D

$$y_{i,j} = \gamma_j \hat{x}_{i,j} + \beta_j$$

Output,
Shape is N x D

Test-time

Input: $x : N \times D$

Learnable scale and shift parameters:

$$\gamma, \beta : D$$

Learning $\gamma = \sigma$,
 $\beta = \mu$ will recover the
identity function!

$\mu_j =$ (Running) average of
values seen during
training

Per-channel
mean, shape is D

$\sigma_j^2 =$ (Running) average of
values seen during
training

Per-channel
std, shape is D

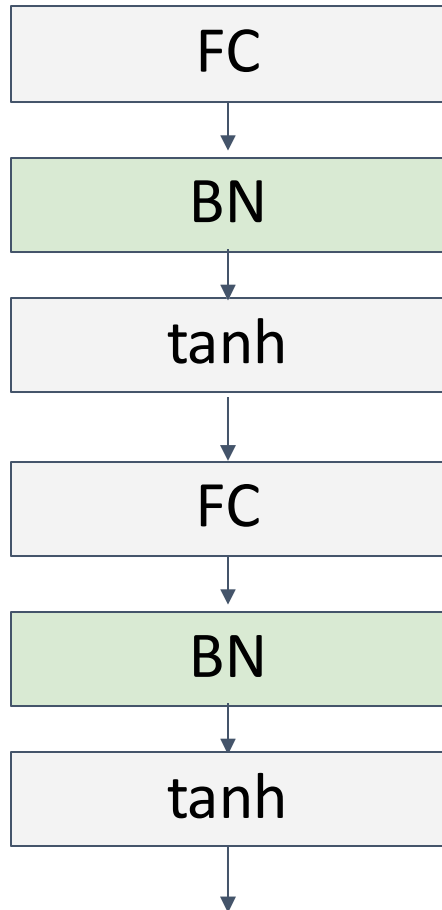
$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \varepsilon}}$$

Normalized x,
Shape is N x D

$$y_{i,j} = \gamma_j \hat{x}_{i,j} + \beta_j$$

Output,
Shape is N x D

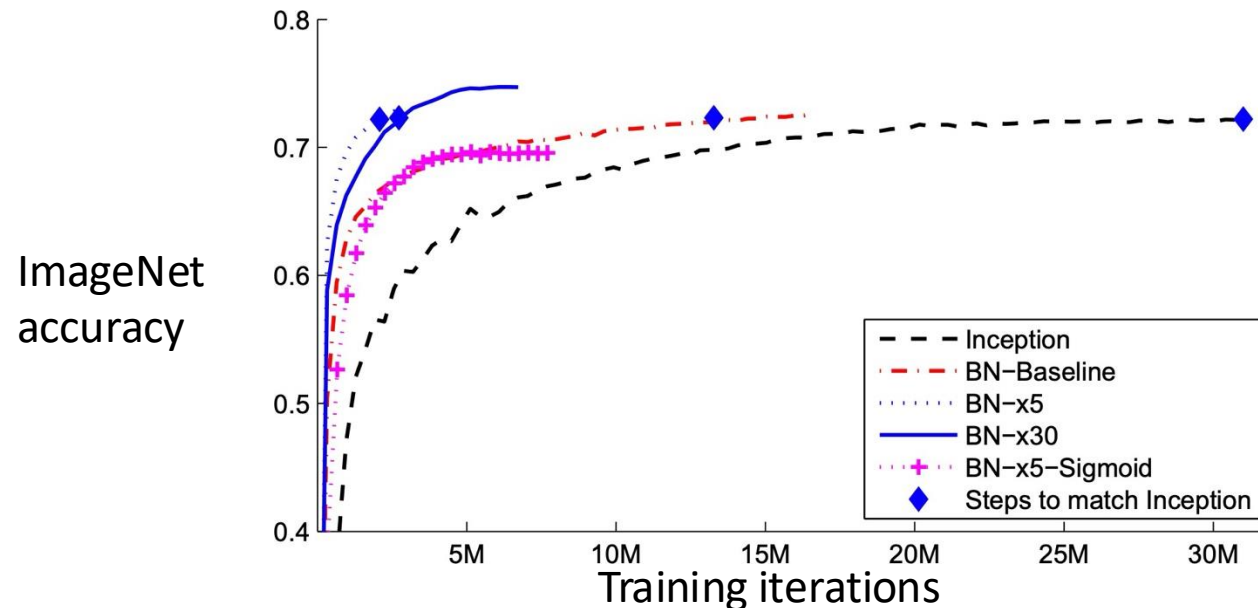
Batch Normalization



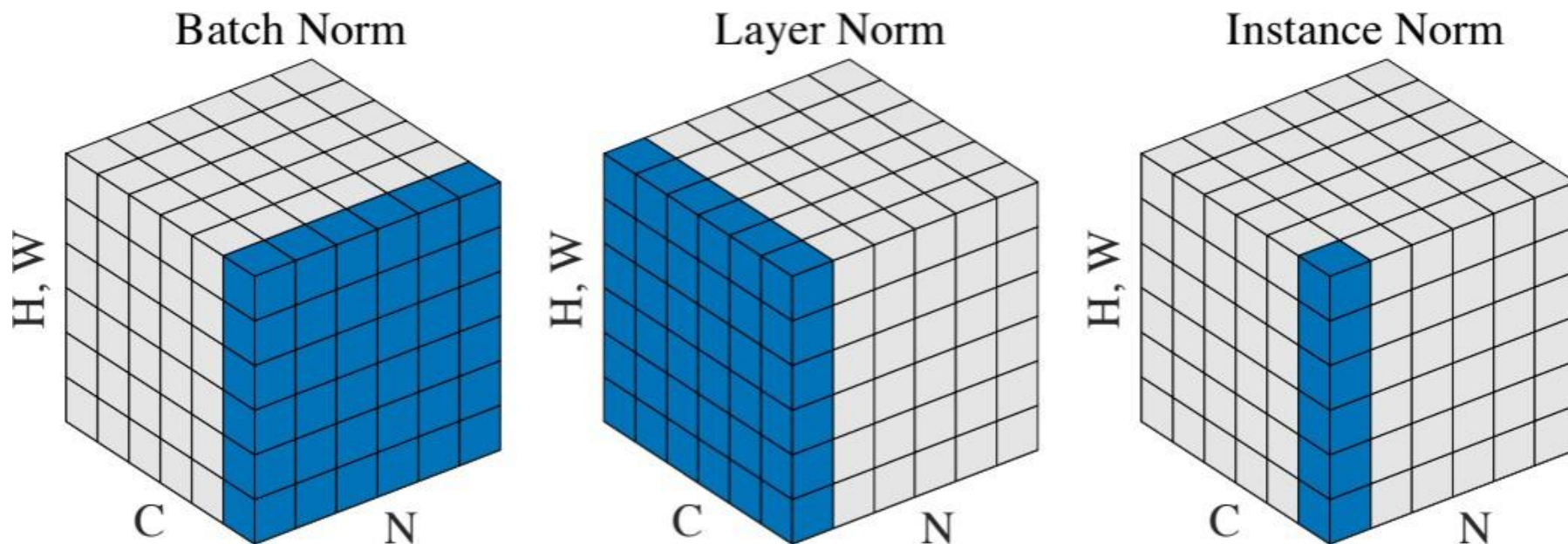
Usually inserted after Fully Connected or Convolutional layers, and before nonlinearity.

Batch Normalization Advantages

- Makes deep networks **much** easier to train!
- Allows higher learning rates, faster convergence
- Networks become more robust to initialization



Other Alternatives of Normalization Layers



Solves the test-train different behaviour issue

Batch Normalization for fully-connected networks

$$\mathbf{x} : \mathbf{N} \times \mathbf{D}$$

Normalize



$$\mu, \sigma : 1 \times \mathbf{D}$$

$$\gamma, \beta : 1 \times \mathbf{D}$$

$$\mathbf{y} = \gamma (\mathbf{x} - \mu) / \sigma + \beta$$

Layer Normalization for fully-connected networks
Same behavior at train and test!
Used in RNNs, Transformers

$$\mathbf{x} : \mathbf{N} \times \mathbf{D}$$

Normalize



$$\mu, \sigma : \mathbf{N} \times 1$$

$$\gamma, \beta : 1 \times \mathbf{D}$$

$$\mathbf{y} = \gamma (\mathbf{x} - \mu) / \sigma + \beta$$

Next time:
Deep Architectures
Representations